

VOFTools

USER MANUAL

Analytical and geometrical tools for
VOF methods in arbitrary grids



UNIVERSIDAD
POLITÉCNICA
DE CARTAGENA

Version 6 | 2026

JOAQUÍN LÓPEZ

This manual is for VOFTools version 6.

VOFTools is a free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. VOFTools is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with VOFTools. If not, see <http://www.gnu.org/licenses/>.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	1
2 Installation	5
2.1 Python users	5
2.1.1 Modifying the default values of <code>ns</code> and <code>nv</code>	5
2.1.2 Executing the test programs	6
2.2 C and Fortran users	7
3 Routines description and usage	11
3.1 Module integration and usage	13
3.1.1 Usage in Fortran	13
3.1.2 Usage in C	13
3.1.3 Usage in Python	14
3.2 Polyhedron and polygon geometries	14
3.3 <code>defpol3d</code> – Initialize polyhedron type	19
3.4 <code>enforv3d</code> – PLIC interface location for arbitrary polyhedra	21
3.5 <code>enforv3dsz</code> – Efficient PLIC interface location for orthogonal hexahe- dral cells	22
3.6 <code>enforvppa</code> – PPIC interface location for arbitrary polyhedra	23
3.7 <code>inte3d</code> – Half-space truncation of arbitrary polyhedra	24
3.8 <code>toolv3d</code> – Volume computation for arbitrary polyhedra	25
3.9 <code>intv3d</code> – Half-space integration volume for arbitrary polyhedra	26
3.10 <code>intc3d</code> – Arithmetic mean of intersection vertices for arbitrary poly- hedra	27
3.11 <code>inte3dface</code> – Half-space intersection for planar polygons	28
3.12 <code>areaface</code> – Area of arbitrary planar polygons	29
3.13 <code>intpv3dpa</code> – Intersection volume between paraboloid and arbitrary polyhedron	30
3.14 <code>box3d</code> – Bounding box computation for point sets	31
3.15 <code>pvfit</code> – Paraboloid fitting for polyhedral interface reconstruction	32
3.16 <code>systra</code> – Rigid-body coordinate transformation	33
3.17 <code>copol3d</code> – Polyhedron deep copy	34

3.18	dist3d – Point-to-convex-polygon Distance in 3D	35
3.19	initf3d – Volume fraction initialization via Cartesian shadow grid . .	36
3.20	polout3d – VTK export for polyhedra	37
3.21	voftools_dim3d – Retrieve array dimensioning parameters for 3D polyhedra	38
3.22	defpol2d – Initialize polygon type	39
3.23	enforv2d – PLIC interface location for 2D polygons	39
3.24	enforv2dsz – Efficient PLIC interface location for rectangular cells . .	40
3.25	inte2d – Half-space truncation of arbitrary polygons	41
3.26	toolv2d – Area computation for arbitrary polygons	42
3.27	cppol2d – Polygon deep copy	43
3.28	dist2d – Point-to-segment distance	44
3.29	initf2d – Area fraction initialization for 2D geometries via Cartesian shadow grid	44
3.30	polout2d – Polygon geometry export in two-column format	46
3.31	voftools_dim2d – Retrieve array dimensioning parameter for 2D polygons	46
3.32	voftoolslogo – Print VOFTools logo	47
4	Verification and validation	49
4.1	PLIC operations	49
4.1.1	Configuration and execution of test programs	49
4.1.2	Runtime workflow analysis and accuracy validation of PLIC operations	60
4.2	PPIC operations	76
4.2.1	Configuration and execution of the fitting operation	79
4.2.2	Configuration and execution of volume truncation and shifting operations	81
	Bibliography	85

List of Figures

3.1	Cell geometries for the 2D cases of Table 3.3	15
3.2	Cell geometries for the 3D cases of Table 3.3	16
3.3	Detailed definition of the non-convex pentagonal pyramid shown in Fig. 3.2 . See the source code of the routine <code>voftools_ncpentapy</code> for parameter definitions. Note that the values of the face normal components (xns , yns , zns) are rounded in the figure for clarity. The red arrow indicates the counterclockwise order of the vertices for face boundary 2 when viewed from the exterior of the polyhedron, consistent with the outward-pointing normal convention.	17
4.1	Positioning the interface by solving the VCE problem on a hexahedral cell.	52
4.2	Truncation of the hexahedral cell by a plane containing the PLIC interface (the intersection vertices are marked with an $\times$ symbol in the left picture).	53
4.3	Computation of the distance from point P to the reconstructed PLIC interface.	53
4.4	Initialization of the volume of a spherical material body contained inside a non-convex cell (example corresponding to <code>icelltype</code> = 113 and <code>ishape</code> = 11).	54
4.5	Illustration of the results produced by the 2D test program for the input data corresponding to those of Table 4.2	55
4.6	Distance d produced by the test program for the input data corresponding to those of Table 4.2 but changing the position of the point P to (a) $(-1, 0)$, (b) $(0.5, 0)$ and (c) $(2, 0)$	56
4.7	Illustration of the results produced by the 2D test program for the input data corresponding to those of Table 4.3	57
4.8	Illustration of the results produced by the 3D test program for the input data corresponding to those of Table 4.4	58
4.9	Distance d produced by the test program for the input data corresponding to those of Table 4.4 but changing the position of the point P to (a) $(0.5, 0, 0.5)$, (b) $(-0.5, 0, 1.5)$ and (c) $(1.5, 0, 0.5)$	59
4.10	Illustration of the results produced by the test program for the input data corresponding to those of Table 4.5	60
4.11	Position coordinates of the vertices of the non-convex pentagonal pyramid shown in Fig. 3.2	61

4.12	Truncation of the polyhedron Ω of Fig. 4.11 (dotted lines) by a plane $\mathcal{P}$ defined by $x_{nc} = 0$, $y_{nc} = -1$, $z_{nc} = 0$ and $c = 0.18$ (highlighted in gray). The truncated polyhedron Ω_T is represented with blue thick lines.	62
4.13	Face discarding.	63
4.14	Clipping procedure for the face boundaries (a) 1, (b) 2, (c) 3, (d) 4 and (e) 5 of Fig. 4.12 . Polygon truncation on the left and truncated polygons, filled with blue (arrows indicate vertex order), on the right.	64
4.15	Sequence (from top to bottom and from left to right) of the capping procedure applied for the construction of the on- $\mathcal{P}$ face boundary (thick lines) in the example of Fig. 4.12	65
4.16	Vertex arrangement of the on- $\mathcal{P}$ face boundary in the example of Fig. 4.12	66
4.17	Example of the truncation between the stellated icosahedron of Fig. 3.2 and a half-space that produces four new on- $\mathcal{P}$ face boundaries (black thick lines). The truncated regions are highlighted in blue.	67
4.18	Sequence of application of the VCE operation in a PLIC reconstruction within the cell of Fig. 4.11 for a case with $x_{nc} = 0$, $y_{nc} = -1$, $z_{nc} = 0$ and $f = v/v_t = 0.55$. (a) First tentative value of the solution and identification of the bracketing interval. (b) Computation of the coefficients of the analytical function $v(c)$ valid inside the tentative bracketing interval, from which the bounds $v_{\max}$ and $v_{\min}$ are obtained, and check if the condition $v_{\min} \leq v < v_{\max}$ is or not satisfied.	68
4.19	Continuation of the sequence of Fig. 4.18 . (a) Steps (i) to (iii) are repeated and the volume condition is checked again. (b) The volume condition is satisfied and the final solution is analytically obtained.	69
4.20	Liquid volume regions (blue color) obtained from the intersection between the VOFTools logo cell (routine <code>voftools_logo</code> ; see Fig. 3.2) and half-spaces defined by planes that contain the PLIC interfaces with normal vector $\mathbf{n} = (0, -1, 0)$ and different liquid volume fractions v/v_t (the solutions of the VCE problem are included in parenthesis). The original cell is depicted on each picture using partial transparent gray.	71
4.21	Same as in Fig. 4.20 but with $\mathbf{n} = (-1, 0, 0)$	72
4.22	Liquid volume fraction $v(c)/v_t$ as a function of c for the VOFTools logo cell and an interface orientation given by $\mathbf{n} = (-1, 0, 0)$	73
4.23	Liquid volume initialization of a circular and spherical liquid body (blue color) with radius $r_l = 0.325$ in the non-convex cell of Fig. 3.1 defined by the routine <code>voftools_hsquare</code> (top picture) and the non-convex cell of Fig. 3.2 defined by the routine <code>voftools_hcube</code> (bottom picture).	74
4.24	Initialization errors as a function of nc for the 2D and 3D cases of Fig. 4.23	76
4.25	Examples for the initialization of the material bodies of Eq. (4.1) (left column) and Eq. (4.2) (right column) in several cells of Fig. 3.1 and Fig. 3.2 , respectively.	77

4.26	Initialization errors as a function of nc for several non-convex cells of Figs. 3.1 and 3.2 and the material bodies with (a) elliptical shape of Eq. (4.1) and (b) toroidal shape of Eq. (4.2), respectively.	78
4.27	Example of the fitting operation performed by the test program <code>testfit</code> . The paraboloid (green surface) is fitted to a set of polygons (magenta lines) defined in a local coordinate system.	80
4.28	Example of the shifting operation performed by the test program <code>testparab</code> . (a) Before shifting: Initial paraboloid (transparent green surface) and cubic cell (transparent gray surfaces) in the local coordinate system (u, v, f) . (b) After shifting: Equivalent result for the shifted paraboloid. The intersection between the paraboloid and the cubic cell boundaries is represented by red lines.	83
4.29	Equivalent to Fig. 4.28 (b) but for (a) tetrahedral (<code>icelltype</code> = 13) and (b) stellated cubic (<code>icelltype</code> = 113) cells, and different fluid volume fractions v/vt	84

List of Tables

3.1	Data type aliases.	11
3.2	Argument intent definitions.	11
3.3	Cell geometries considered in the test programs of Section 4.	12
3.4	Values of the index used to denote the cell geometry routines considered in the test programs.	20
4.1	Values of the <code>ishape</code> index used to denote the body shapes considered in the test programs.	51
4.2	Input data for example 1.	54
4.3	Input data for example 2.	55
4.4	Input data for example 3.	57
4.5	Input data for example 4.	59
4.6	Array <code>ipv0</code> of vertex number, i_p , assigned to every vertex index i of face boundary j of the polyhedron of Fig. 4.11 . Each table entry is <code>ipv0(j,i)</code>	61
4.7	Array <code>ipv1</code> of the index number, i_p , assigned to every vertex index i of face boundary j of the polyhedron, Ω_T , resulting from the truncation operation of Fig. 4.12 . Each table entry is <code>ipv1(j,i)</code>	63

1 — Introduction

`VOFTools` is a software package designed to provide analytical and geometrical tools for 2D/3D volume-of-fluid (VOF) methods in arbitrary grids. It supports both convex or non-convex cell types, which are common in complex computational domains. The library includes efficient algorithms for area/volume computation, truncation operations frequently encountered in VOF methods, and area/volume conservation enforcement (VCE) to accurately locate interfaces in piecewise linear interface calculation (PLIC) and piecewise paraboloid interface calculation (PPIC) reconstructions. Additionally, `VOFTools` supports liquid area/volume fraction initialization and computes the shortest distance from a given point to the reconstructed PLIC interface. Previous versions of this library have been described in [6–9, 11, 12].

The current version has been extensively revised and restructured to enhance interoperability through the `ISO_C_BINDING` module. This new version incorporates several improvements in both accuracy and computational efficiency for handling intersections between polyhedra and half-spaces, as well as optimized methods for computing the interface position to cut off a given area or volume fraction from an arbitrary polygonal/polyhedral grid cell during PLIC reconstruction. Furthermore, the updated codebase includes new routines designed specifically for:

1. Paraboloid reconstruction via volumetric-error-minimization fitting of discrete polygonal data.
2. Computation of the intersection volume between the PPIC fluid domain and an arbitrary polyhedral cell.
3. Conservative PPIC reconstruction for interface localization in arbitrary polyhedral cells.
4. Efficient computation of the average position of clipped vertices and the resulting volume arising from the intersection of an arbitrary polyhedron with a half-space. This routine is particularly advantageous for PLIC reconstruction algorithms and unsplit advection schemes based on fluid-volume intersection advection (FVIA).
5. Three-dimensional clipping of an arbitrary oriented planar polygon by a half-space, including the computation of the resulting area. This feature is essential for unsplit advection schemes based on wetted-area time-integration advection (WATIA).

6. Auxiliary computational utilities for robust simulation setup, including geometric bounding-box estimation and dynamic memory configuration.

These enhancements significantly improve the code's overall functionality, robustness, and computational efficiency in handling complex geometric calculations critical to advanced VOF methods. Comprehensive implementation details and performance analyses are provided in [13–15].

VOFTools is implemented in Fortran, with C and Python (v3) interfaces provided to facilitate interoperability. The Python bindings utilize the `f2py` module from the NumPy library [19]. To promote reproducibility and ease of access, VOFTools is distributed via the Python Package Index (PyPI) [18].

The package includes the following key components:

- `src`: Directory containing the core source code and interface definitions.
 - `dimpol.h`: Header file declaring parameters `ns` and `nv` used for pre-allocating arrays in polytope computations.
 - `voftools.f90`: Fortran source code implementing the core routines of the VOFTools library.
 - `uservoftools.f90`: Fortran source file containing user-defined routines and functions, allowing for custom logic integration.
 - `cvoftools.h`: C header file declaring the interfaces for the core VOFTools library routines, enabling use in C programs.
 - `cuservoftools.h`: C header file exposing the user-defined routines from `uservoftools.f90` for integration into C applications.
 - `voftools.pyf`: F2PY signature file that defines how Fortran routines in `voftools.f90` are exposed to Python, facilitating binary interface compatibility.
 - `cellparabout.py`: Python script that reads a polyhedron from a VTK file and a set of paraboloid coefficients from a text file. It then generates the corresponding paraboloid surface, computes the intersection curve between the polyhedron and the paraboloid, and visualizes the results using `pyvista`.
- `__init__.py`: Python package initialization file, enabling the directory to be imported as a module.
- `.f2py_f2cmap`: Configuration file for NumPy's `f2py` tool, mapping Fortran data types to corresponding C types to ensure correct memory layout and data transfer.
- `examples`: Directory containing test programs demonstrating library usage in Python, C, and Fortran. Each language example directory includes:
 - `test2d`: Script for 2D PLIC operations.

- `test3d`: Script for 3D PLIC operations.
 - `testfit`: Script for PPIC fitting operation.
 - `testparab`: Script for PPIC truncation and shifting operations.
- `data`: Directory containing static input data files (e.g., polygon sets) required by the test programs.
- `vofvardef`: Input configuration file providing parameters for the C and Fortran test executables.
- `Makefile`: Build script that automates the compilation of the VOFTools library and the execution of C/Fortran test programs.
- `meson.build`: Meson build system configuration file defining dependencies, compiler flags, and installation rules for the VOFTools library.
- `pyproject.toml`: Python packaging configuration that specifies metadata, dependencies, and build settings, supporting Python versions 3.11-3.14 across Linux, macOS, and Windows.
- `doc`: Directory containing the user manual in PDF format.
 - `user-manual-voftools-6.pdf`.
- `readme.md`: Project overview file.
- `LICENSE`: Copy of the GNU General Public License (GPL), Version 3.

Note: The index notation used in the illustrative examples throughout this manual follows the convention of starting from 1, which is consistent with Fortran array indexing. In contrast, C and Python use zero-based indexing. Users should be aware of this distinction when interpreting array indices in the provided examples.

2 — Installation

To install the `VOFTools` library and run the provided test programs in Fortran, C, and Python, follow the steps outlined below.

Note that the core routines utilize pre-allocated arrays for face indices (`ns`) and vertex indices (`nv`), with default limits of 140 and 200, respectively. While these limits are sufficient for many applications, including computational fluid dynamics (CFD), users requiring support for significantly higher-complexity polyhedral cells may need to adjust the default values.

2.1 Python users

Python 3.11-3.14 is supported. Using a virtual environment is recommended to isolate dependencies.

Install `VOFTools` from PyPI using:

```
python -m pip install voftools
```

When a compatible precompiled package (wheel) is available for the Python version, operating system, and processor architecture, pip installs it without requiring C or Fortran compilers. Otherwise, pip attempts to build the package from the source distribution. Building from source requires `GCC` and `GFortran`, together with the corresponding system development tools. Python build dependencies are installed automatically in an isolated build environment.

`NumPy` is a required dependency and is installed automatically by pip. To run examples that use `PyVista` for visualization, also install:

```
python -m pip install pyvista
```

To check that the library can be imported and query its default array dimensions, run:

```
python -c "import voftools as v; print(v.voftools_dim3d())"
```

For the default configuration, the expected output is `'(140, 200)'`.

2.1.1 Modifying the default values of `ns` and `nv`

The parameters `'ns'` and `'nv'` are fixed when the library is compiled. Changing them therefore requires rebuilding the Python extension from source.

Download and extract the source package, then open a terminal in the extracted directory containing 'pyproject.toml'. GCC and GFortran must be installed and available to the build system. Python build dependencies are installed automatically in an isolated environment.

Edit 'src/dimpol.h' to set the required dimensions. For example:

```
INTEGER, PARAMETER :: NS=240, NV=300
```

Alternatively, on systems with GNU Make and a compatible shell, the same file can be generated using:

```
make dim NS=240 NV=300
```

This command updates the dimension definitions; it does not build or install the Python extension.

With the intended Python environment active and NumPy already installed as described in Section 2.1, rebuild and reinstall VOFTools from the source directory:

```
python -m pip install --force-reinstall --no-deps --no-cache-dir .
```

The `--no-deps` option preserves the runtime dependencies already installed in the environment. Build dependencies are still managed automatically.

Verify the dimensions in a new Python process:

```
python -c "import voftools as v; print(v.voftools_dim3d())"
```

For the values used above, the expected output is '(240, 300)'. Restart any Python sessions or notebook kernels that had already imported VOFTools before using the rebuilt library.

2.1.2 Executing the test programs

The Python examples are provided in the 'examples/python/' directory of the source package. Installing VOFTools with pip does not place these scripts in the current working directory.

To run the examples, extract the source package and open a terminal in its root directory. Activate the Python environment in which VOFTools is installed. Before running 'testfit.py', copy 'data/polygons-set.txt' to this working directory.

Run the examples using:

```
python examples/python/test2d.py
python examples/python/test3d.py
python examples/python/testfit.py
python examples/python/testparab.py
```

Input parameters for the Python examples are defined directly in the corresponding scripts. The 'vofvardef' configuration file is used by the C and Fortran examples 'test2d' and 'test3d'.

Output files are written to the current working directory. The operations and expected results are described in Chapter 4.

The 'test3d.py' example uses PyVista to display the original and truncated cells. Close each visualization window to allow execution to continue. In environments

without a compatible graphical display, the two `'grid.plot(...)'` calls can be commented out to run the numerical calculations without opening windows. PyVista is still required by the remaining visualization-related code.

2.2 C and Fortran users

The source package includes a `Makefile` for building the static library and the C and Fortran examples. It uses `GCC` and `GFortran` by default and requires GNU Make and a compatible shell. On Windows, an appropriate build environment must be configured; compiler names, paths, and options may need to be adapted.

1. **Extract the source package in the chosen working directory:**

```
tar -zxvf voftools-6.tgz
```

2. **Enter the extracted directory:**

```
cd voftools-6
```

3. **Build the library and example programs:**

```
make
```

The compiler commands can be specified explicitly. For example:

```
make CC=gcc F77=gfortran
```

Replace `gcc` and `gfortran` with the appropriate executable names or paths if necessary. Compiler-specific options in the `Makefile` must also be reviewed when using a different compiler family.

To use dimensions other than the defaults, specify both parameters:

```
make NS=240 NV=300
```

This sets `ns` to 240 and `nv` to 300 in `src/dimpo1.h` before compilation. When invoking `make` without these assignments, the `Makefile` uses its default values of 140 and 200.

The build creates the static library `lib/libvoftools.a` and the following executables in the package root directory:

- **C:** `test2d_c`, `test3d_c`, `testfit_c`, and `testparab_c`.
- **Fortran:** `test2d_f`, `test3d_f`, `testfit_f`, and `testparab_f`.

To clean the compiled files before rebuilding, run:

```
make clean
```

Then repeat the build command, including any custom compiler settings and dimension values.

4. **Configure input parameters:** Before running `test2d_c`, `test2d_f`, `test3d_c`, or `test3d_f`, edit the `vofvardef` file in the working directory to select the desired case. The 2D and 3D programs read the same file, so its values must be adjusted when switching between cases. Parameter descriptions are provided in the example source files and in Chapter 4.

The `testfit` and `testparab` programs do not use `vofvardef`; their configuration is described in Section 4.2.

Example configuration: The following configuration corresponds to a 3D test case involving:

- A half-space defined by the unit normal vector ($xnc = 0$; $ync = -1$; $znc = 0$).
- Intersection with a cubic cell (`icelltype = 11`) having a truncated volume fraction $f = 0.5$.
- Computation of the distance from point P ($x_p = 0$; $y_p = 0$; $z_p = 0$) to the reconstructed PLIC interface. This distance calculation requires a convex interfacial polygon. The example program performs it only for the selected convex cell types and when the cutting plane leaves nonzero portions of the cell on both sides.
- Estimation of the volume of a spherical material body (`ishape = 11`) within the cell using the initialization procedure with `nc=10` subdivisions per coordinate direction and tolerance `tol=10`.

```
Cell geometry, ICELLTYPE:
11
Material body shape, ISHAPE:
11
Material volume/area fraction, F:
0.5
X coordinate of the unit-length normal vector of the interface plane,
  XNC:
0.0
Y coordinate of the unit-length normal vector of the interface plane,
  YNC:
-1.0
Z coordinate of the unit-length normal vector of the interface plane,
  ZNC:
0.0          !Ignore for 2D test programs
X coordinate of point P from which the distance is calculated, XP:
0.0
Y coordinate of point P from which the distance is calculated, YP:
0.0
Z coordinate of point P from which the distance is calculated, ZP:
0.0          !Ignore for 2D test programs
Subdivision number in the volume fraction cell initialization, NC:
10
Tolerance for the initialization procedure, TOL:
10.0
```

5. **Execute a test program:** Run the executable from the package root directory, where the required input files are located. For example, to run the 3D Fortran test:

```
./test3d_f
```

To run the corresponding C test:

```
./test3d_c
```

The `testfit` executables require `polygons-set.txt` in the working directory; the `Makefile` copies this file from `data/` during the build. Output files are written to the working directory. See Chapter 4 for the configuration and interpretation of each example.

3 — Routines description and usage

V0FTools utilizes the ISO_C_BINDING standard to ensure compatibility across Fortran, C, and Python interfaces. Table 3.1 defines the aliases for data types used through-

Table 3.1: Data type aliases.

Alias	Fortran	C	Python
W_P	REAL(C_DOUBLE)	double	float64
I_P	INTEGER(C_INT)	int	int32
I_P2	INTEGER(C_SHORT)	short	int16

out this manual.

To clarify data handling during subroutine calls, the argument intent labels in Table 3.2 are employed¹.

Table 3.2: Argument intent definitions.

Label	Definition	Behavior
IN	Input	Read-only upon entry; remains unchanged by the routine
OUT	Output	Set by the routine upon return; entry values are ignored
INOUT	Input/Output	Modified in-place; read by the routine and subsequently updated

Once the installation is complete, the V0FTools routines and functions can subsequently be used in Fortran, C, and Python programs as described below (see the test programs provided in the supplied software).

¹Python users must pass the INOUT parameters as NumPy arrays to be modified in-place.

Table 3.3: Cell geometries considered in the test programs of Section 4.

Routine name	Cell geometry
3D	
voftools_cube	Cube
voftools_ihexa	Irregular hexahedron
voftools_tetra	Tetrahedron
voftools_dodeca	Dodecahedron
voftools_icos	Icosahedron
voftools_ipol3d	Complex polyhedron with 32 vertices and 18 faces
voftools_ncpentapy	Non-convex pentagonal pyramid
voftools_nccubepy	Non-convex cell obtained by subtracting a pyramid to the cubic cell
voftools_scube	Stellated cube
voftools_nchexa	Non-convex hexahedron
voftools_sdodeca	Stellated dodecahedron
voftools_sicosa	Stellated icosahedron
voftools_hcube	Hollowed cube
voftools_drcube	Drilled cube
voftools_zigzag	Zig zag prism
voftools_logo	VOFTools logo
2D	
voftools_square	Square
voftools_hexagon	Regular hexagon
voftools_tri	Irregular triangle
voftools_quad	Irregular quadrangle
voftools_pentagon	Irregular pentagon
voftools_ihexagon	Irregular hexagon
voftools_ncquad	Non-convex quadrangle
voftools_ncpentagon	Non-convex pentagon
voftools_nchexagon	Non-convex hexagon
voftools_shexagon	Stellated hexagon
voftools_hsquare	Hollowed square
voftools_msquare	Non-convex multi-square cell

3.1 Module integration and usage

VOFTools provides the Fortran modules `voftools_mod` and `uservoftools_mod`. Their corresponding `.mod` files are generated during compilation. For C interoperability, the header files `cvoftools.h` and `cuservoftools.h` provide the declarations needed to call the Fortran routines from C programs.

3.1.1 Usage in Fortran

To utilize VOFTools within a Fortran program, include the appropriate modules as shown in Listing 3.1.

```
program fortran_example
  use voftools_mod !- Main VOFTools module
  use uservoftools_mod !- User-defined routines module
  use, intrinsic :: iso_c_binding, only: w_p => c_double, i_p => c_int
  implicit none
  ! Declare variables using the kind aliases defined above
  ! Invoke the routines and functions, e.g.:
    call voftoolslogo
  !... other routine calls
end program fortran_example
```

Listing 3.1: Fortran invocation example.

3.1.2 Usage in C

To utilize VOFTools within a C program, include the provided headers as shown in Listing 3.2.

```
#include <stdio.h>
#include "cvoftools.h" // Header for core VOFTools routines
#include "cuservoftools.h" // Header for user-defined routines
int main() {
  // Retrieve array dimensions
  int ns, nv;
  voftools_dim3d(&ns, &nv);
  // Declare required variables
  // Invoke the routines and functions, e.g.:
  voftoolslogo();
  //... other routine calls
  return 0;
}
```

Listing 3.2: C invocation example.

The header `cvoftools.h` declares the core VOFTools routines. Include the header `cuservoftools.h` when using routines from `uservoftools_mod`. Compilation and linking follow the procedure described in Section 2.2.

3.1.3 Usage in Python

VOFTTools is exposed to Python via the `voftools` package, which wraps the underlying Fortran routines using `f2py`. Users can import the module as shown in Listing 3.3. Use NumPy arrays with the shapes and data types required by each routine. Multidimensional arrays passed as INOUT arguments must be writable and Fortran-contiguous so that `f2py` can modify them in place. Such arrays can be allocated with `order='F'`. For input-only arrays, `f2py` may create a temporary copy to convert the data type or memory layout required by the Fortran routine. For efficiency, create multidimensional input arrays in Fortran order with the required data type, particularly when they are reused across multiple routine calls. This avoids unnecessary internal copies.

```
import numpy as np
from voftools import voftools_mod as voft
#. Retrieve array dimensions
(ns,nv) = voft.voftools_dim3d()
#. Invoke the routines and functions, e.g.:
voft.voftoolslogo()
#... other routine calls
```

Listing 3.3: Python invocation example.

Note: In what follows, it is assumed that the `voftools_mod` module has been loaded as indicated in the previous example.

3.2 Polyhedron and polygon geometries

VOFTTools allows for the representation of arbitrary convex and non-convex polyhedral and polygonal geometries. While Table 3.3 summarizes the specific cell types implemented in the test programs of Section 4, the core library enables users to define arbitrary polytopes beyond this predefined set. These geometries are defined by invoking specific initialization routines that return the necessary geometric parameters.

The geometry of polyhedra and polygons is represented using connectivity arrays and coordinate lists. To maintain compatibility with fixed-dimension buffers and avoid jagged array complexities, these quantities are physically allocated with a maximum capacity `ns` (for faces) or `nv` (for vertices). However, their logical sizes correspond to the actual number of entities (e.g., `nts` for face boundaries and `ntv` for vertices). This convention ensures that routines operate on logical arrays of the appropriate size while respecting the allocated physical buffers of size `ns` or `nv`.

Figs. 3.1 and 3.2 illustrate representative cells using the `polout2d` (Section 3.30) and `polout3d` (Section 3.20) visualization routines in conjunction with Gnuplot [16] and ParaView [3], respectively. The routine name corresponding to each geometry is indicated within the figures.

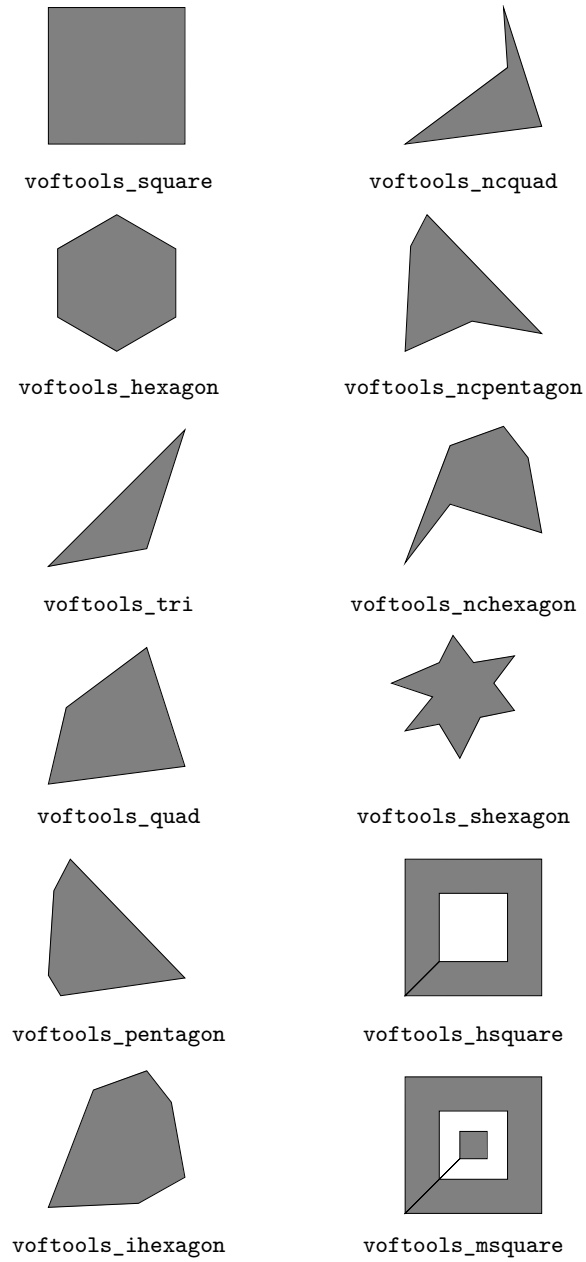


Figure 3.1: Cell geometries for the 2D cases of Table 3.3.

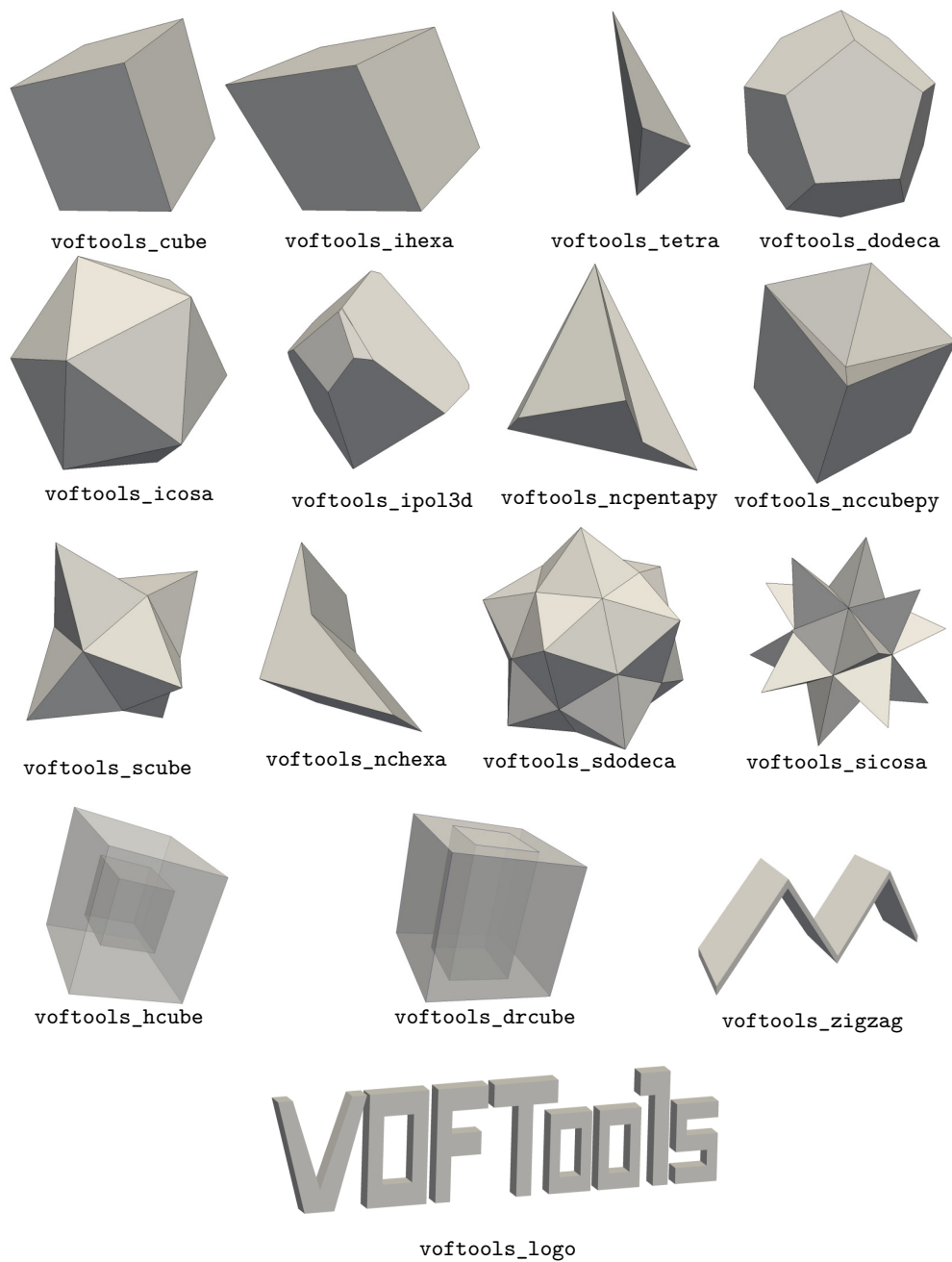


Figure 3.2: Cell geometries for the 3D cases of Table 3.3.

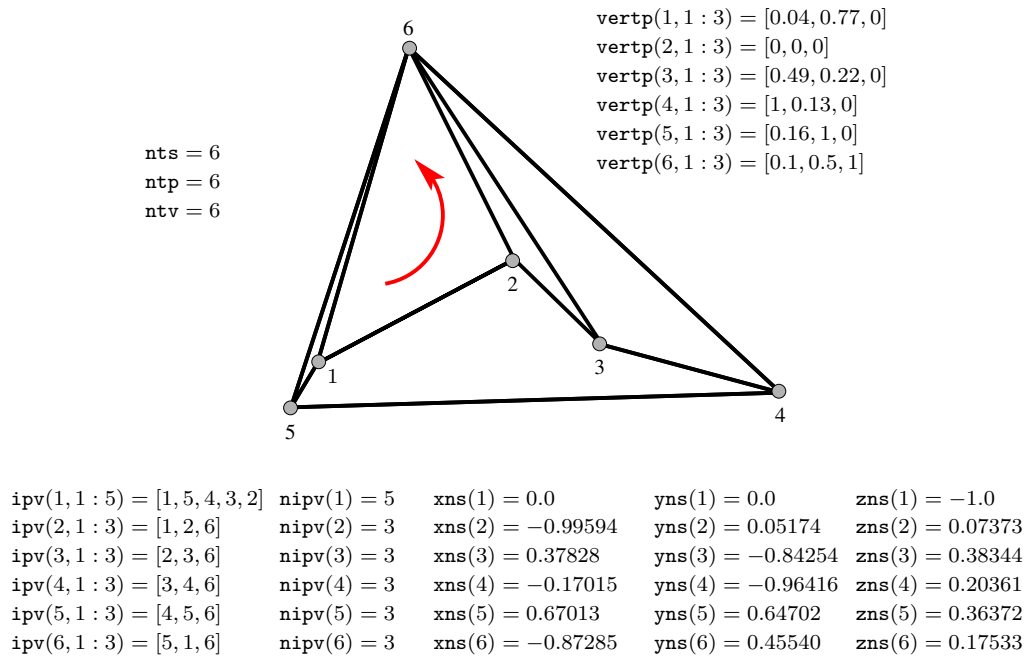


Figure 3.3: Detailed definition of the non-convex pentagonal pyramid shown in Fig. 3.2. See the source code of the routine `voftools_ncpentapy` for parameter definitions. Note that the values of the face normal components (`xns`, `yns`, `zns`) are rounded in the figure for clarity. The red arrow indicates the counterclockwise order of the vertices for face boundary 2 when viewed from the exterior of the polyhedron, consistent with the outward-pointing normal convention.

As an example, the calling conventions for a cubic cell are provided below for Fortran (blue background), C (green background) and Python (red background):

Fortran:

```
call voftools_cube(ipv, nipv, ntp, nts, ntv, vertp, xns, yns, zns)
```

C:

```
voftools_cube(ipv, nipv, &ntp, &nts, &ntv, vertp, xns, yns, zns);
```

Python:

```
(ipv, nipv, ntp, nts, ntv, vertp, xns, yns, zns) = voftools_cube()
```

Arguments:

Argument	Intent	Type	Definition
ipv	OUT	I_P	array of dimensions (ns, nv) storing the vertex indices defining each face boundary
nipv	OUT	I_P	array of length ns storing the number of vertices per face boundary
ntp	OUT	I_P	maximum vertex index in the connectivity list
nts	OUT	I_P	total number of face boundaries
ntv	OUT	I_P	total number of vertices
vertp	OUT	W_P	array of dimensions $(nv, 3)$ storing the x, y, z -coordinates of each vertex
xns, yns, zns	OUT	W_P	arrays of length ns storing the x, y, z -components of the outward-pointing unit normal vectors for each face boundary

To illustrate a specific example, Fig. 3.3 depicts the non-convex pentagonal pyramid defined by the supplied routine `voftools_ncpentapy` (also shown in Fig. 3.2).

For a square cell,

Fortran:

```
call voftools_square(ipv,ntp,ntv,vertp)
```

C:

```
voftools_square(ipv,&ntp,&ntv,vertp);
```

Python:

```
(ipv,ntp,ntv,vertp) = voft.voftools_square()
```

Arguments:

Argument	Intent	Type	Definition
ipv	OUT	I_P	array of length nv storing the vertex indices defining the polygon
ntp	OUT	I_P	maximum vertex index in the connectivity list
ntv	OUT	I_P	total number of vertices
vertp	OUT	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex

Note 1: Initially, $ntp = ntv$ for the defined polytope. However, if subsequent intersection operations are performed on the polytope, these values may diverge for the resulting truncated geometry. This is because ntp tracks the maximum vertex index in the connectivity list, while ntv counts the total count of unique vertices.

Note 2: For detailed definitions of these parameters and their specific roles within the example geometry-defining routines, please refer to the source code of each respective routine.

Similarly, the arguments and calling conventions for the remaining routines in the VOFTools library (voftools.f90) are described in detail below.

To distinguish variables corresponding to different states of a geometric entity (e.g., before and after an operation), numerical suffixes are appended to the parameter names.

3.3 defpol3d – Initialize polyhedron type

The defpol3d routine selects one of the polyhedral geometries shown in Fig. 3.2, determined by the integer parameter icelltype (see Table 3.4). The calling conventions for this routine in Fortran, C, and Python are as follows:

Fortran:

```
call defpol3d(icelltype,ipv,nipv,ntp,nts,ntv,vertp,xns,yns,zns)
```

C:

```
defpol3d(&icelltype,ipv,nipv,&ntp,&nts,&ntv,vertp,xns,yns,zns);
```

Python:

```
(ipv,nipv,ntp,nts,ntv,vertp,xns,yns,zns)=voft.defpol3d(icelltype)
```

Arguments:

icelltype	IN	I_P	cell geometry index
ipv	OUT	I_P	array of dimensions (ns,nv) storing the vertex indices defining each face boundary
nipv	OUT	I_P	array of length ns storing the number of vertices per face boundary
ntp	OUT	I_P	maximum vertex index in the connectivity list
nts	OUT	I_P	total number of face boundaries
ntv	OUT	I_P	total number of vertices
vertp	OUT	W_P	array of dimensions (nv,3) storing the x,y,z-coordinates of each vertex
xns, yns, zns	OUT	W_P	arrays of length ns storing the x,y,z-components of the outward-pointing unit normal vectors for each face boundary

Note: The defpol3d routine, used by the test3d program to select predefined cells, accepts icelltype values 11–16 for convex cells and 111–120 for non-convex cells. This selection does not restrict users to these geometries: user-defined polyhedral cells can be supplied directly to the relevant VOFTools routines through their vertex

Table 3.4: Values of the index used to denote the cell geometry routines considered in the test programs.

Routine name	icelltype index
<i>3D geometries</i>	
voftools_cube	11
voftools_ihexa	12
voftools_tetra	13
voftools_dodeca	14
voftools_icoso	15
voftools_ipol3d	16
voftools_ncpentapy	111
voftools_nccubepy	112
voftools_scube	113
voftools_nchexa	114
voftools_sdodeca	115
voftools_sicosa	116
voftools_hcube	117
voftools_drcube	118
voftools_zigzag	119
voftools_logo	120
<i>2D geometries</i>	
voftools_square	1
voftools_hexagon	2
voftools_tri	3
voftools_quad	4
voftools_pentagon	5
voftools_ihexagon	6
voftools_ncquad	101
voftools_ncpentagon	102
voftools_nchexagon	103
voftools_shexagon	104
voftools_hsquare	105
voftools_msquare	106

coordinates, face connectivity, and outward-pointing unit face normals, subject to each routine's requirements and the compiled array capacities. An invalid `icelltype` value causes `defpol3d` to print a diagnostic message and terminate the program through a Fortran `STOP` statement, including when called from C or Python.

3.4 enforv3d – PLIC interface location for arbitrary polyhedra

The `enforv3d` routine addresses the VCE problem in 3D, specifically aiming to locate the PLIC interface within a polyhedral cell. The subroutine can be invoked in Fortran, C, and Python as follows:

Fortran:

```
call enforv3d(c,ipv,nipv,ntp,nts,ntv,v,vt,vertp,xnc,xns,&
             ync,yns,znc,zns)
```

C:

```
enforv3d(&c,ipv,nipv,&ntp,&nts,&ntv,&v,&vt,vertp,&xnc,xns,&
        ync,yns,&znc,zns);
```

Python:

```
c = voft.enforv3d(ipv,nipv,ntp,nts,ntv,v,vt,vertp,xnc,xns,
                 ync,yns,znc,zns)
```

Arguments:

<code>c</code>	OUT	W_P	scalar representing the signed distance from the PLIC interface to the origin along the normal vector $\mathbf{n}$
<code>ipv</code>	IN	I_P	array of dimensions (ns,nv) storing the vertex indices defining each face boundary
<code>nipv</code>	IN	I_P	array of length <code>ns</code> storing the number of vertices per face boundary
<code>ntp</code>	IN	I_P	maximum vertex index in the connectivity list
<code>nts</code>	IN	I_P	total number of face boundaries
<code>ntv</code>	IN	I_P	total number of vertices
<code>v</code>	IN	W_P	scalar representing the target liquid volume within the cell
<code>vt</code>	IN	W_P	scalar representing the total volume of the polyhedron
<code>vertp</code>	IN	W_P	array of dimensions $(nv,3)$ storing the x,y,z -coordinates of each vertex
<code>xnc, ync, znc</code>	IN	W_P	x,y,z -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

`xns, yns, zns` IN W_P arrays of length `ns` storing the x, y, z -components of the outward-pointing unit normal vector for each face boundary

Note: This routine is intended for the original, untruncated cell geometry, with (`ntp = ntv`). The total cell volume must satisfy $vt > 0$, and the target liquid volume must satisfy $0 \leq v \leq vt$. The argument `v` is a volume, not a volume fraction; for a prescribed liquid volume fraction α , use $v = \alpha vt$. If $vt \leq 0$ or `ntp > ntv`, the routine prints a diagnostic message and returns without assigning a valid value to `c`.

3.5 `enforv3dsz` – Efficient PLIC interface location for orthogonal hexahedral cells

The `enforv3dsz` routine addresses the VCE problem within rectangular parallelepipedic cells, such as those defined in the `voftools_cube` routine. It employs the more efficient analytical method proposed by Scardovelli and Zaleski [20], which is specifically designed for use with this type of cell.

Fortran:

```
call enforv3dsz(c,dx,dy,dz,v,vertp,xnc,ync,znc)
```

C:

```
enforv3dsz(&c,&dx,&dy,&dz,&v,vertp,&xnc,&ync,&znc);
```

Python:

```
v = np.array(v, dtype=np.float64)
c = voft.enforv3dsz(dx,dy,dz,v,vertp,xnc,ync,znc)
```

Arguments:

<code>c</code>	OUT	W_P	scalar representing the signed distance from the PLIC interface to the origin along the normal vector $\mathbf{n}$
<code>dx, dy, dz</code>	IN	W_P	cell dimensions along the coordinate axes x, y, z
<code>v</code>	INOUT	W_P	scalar representing the target liquid volume within the cell; modified internally and restored to its original value before return
<code>vertp</code>	IN	W_P	array of dimensions $(nv, 3)$ storing the x, y, z -coordinates of each vertex
<code>xnc, ync, znc</code>	IN	W_P	x, y, z -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

Note: The cell must be a rectangular parallelepiped with edges parallel to the coordinate axes. Its eight vertices must occupy the first eight rows of `vertp`, and `dx`, `dy`, and `dz` must be the corresponding positive side lengths. The target liquid volume must satisfy $0 \leq v \leq dx dy dz$.

3.6 enforvppa – PPIC interface location for arbitrary polyhedra

The `enforvppa` routine addresses the VCE problem in 3D, specifically aiming to locate the PPIC interface within an arbitrary polyhedral cell. The PPIC interface is defined within a local reference system (u, v, f) as:

$$f = c_1 + c_2u + c_3v + c_4u^2 + c_5uv + c_6v^2, \quad (3.1)$$

where the coefficient $c_1 = c$ is computed by the routine to ensure that the volume of liquid within the polyhedral cell matches the target volume vf . This calculation employs an improved version of the paraboloid-polyhedral approximation procedure based on recursive local grid refinement of the polyhedral cell proposed by López [14].

Fortran:

```
call enforvppa(c, cparab, ie, ipv, nc, nipv, ntp, nts, vf, vt, vertp, &
              xns, yns, zns)
```

C:

```
enforvppa(&c, cparab, &ie, ipv, &nc, nipv, &ntp, &nts,
          &vf, &vt, vertp, xns, yns, zns);
```

Python:

```
(c, ie) = voft.enforvppa(cparab, ipv, nc, nipv, ntp, nts, vf, vt, vertp, xns,
                        yns, zns)
```

Arguments:

<code>c</code>	OUT	W_P	scalar corresponding to the coefficient c_1 in Eq. (3.1), representing the conservative position of the paraboloid interface along the local f -axis
<code>cparab</code>	IN	W_P	12-component array containing: the six paraboloid coefficients defining the surface geometry (see Eq. 3.1, indices 1-6); the three components of the unit vector along the local f -axis, expressed in global coordinates (indices 7-9); and the three coordinates of the local origin in global coordinates (indices 10-12)
<code>ie</code>	OUT	I_P	integer status flag: 0 indicates successful convergence; 1 indicates failure to find a root
<code>ipv</code>	IN	I_P	array of dimensions (<code>ns</code> , <code>nv</code>) storing the vertex indices defining each face boundary.
<code>nc</code>	IN	I_P	scalar specifying the number of sub-cells along each coordinate axis of the superimposed Cartesian grid used for the refinement process
<code>nipv</code>	IN	I_P	array of length <code>ns</code> storing the number of vertices per face boundary

<code>ntp</code>	IN	I_P	maximum vertex index in the connectivity list
<code>nts</code>	IN	I_P	total number of face boundaries
<code>vf</code>	IN	W_P	scalar representing the target liquid volume within the cell
<code>vt</code>	IN	W_P	scalar representing the total volume of the polyhedron
<code>vertp</code>	IN	W_P	array of dimensions $(nv,3)$ storing the x,y,z -coordinates of each vertex
<code>xns, yns, zns</code>	IN	W_P	arrays of length <code>ns</code> storing the x,y,z -components of the outward-pointing unit normal vector for each face boundary

Note: The first component of `cparab` provides the initial estimate of c_1 . The routine returns the computed coefficient in `c` without modifying `cparab`. After checking that `ie = 0`, update `cparab(1)` in Fortran or `cparab[0]` in C and Python before using the reconstructed interface in subsequent calculations. The component indices in the argument description follow Fortran's one-based indexing.

The routine assumes the original, untruncated cell geometry, with contiguous vertex indices from 1 to `ntp`. The subdivision parameter `nc` must be a positive integer. The total cell volume must satisfy `vt > 0`, and the target liquid volume must satisfy $0 \leq vf \leq vt$. Here, `vf` denotes a volume, not a volume fraction.

Always check `ie` before using `c`. If `ie = 1`, the routine has not obtained an acceptable solution, and `c` must not be used as a successfully reconstructed interface.

3.7 `inte3d` – Half-space truncation of arbitrary polyhedra

The `inte3d` routine performs geometric clipping of an arbitrary polyhedron against a half-space defined by the plane $\mathbf{n} \cdot \mathbf{x} + c = 0$. It retains the portion of the polyhedron lying on the positive side of the plane (i.e., where $\mathbf{n} \cdot \mathbf{x} + c > 0$, consistent with the liquid phase convention) and discards the portion on the negative side.

Fortran:

```
call inte3d(c,icontn,icontp,ipv,nipv,ntp,nts,ntv,vertp, &
           xnc,xns,ync,yns,znc,zns)
```

C:

```
inte3d(&c,&icontn,&icontp,ipv,nipv,&ntp,&nts,&ntv,vertp,
&xnc,xns,&ync,yns,&znc,zns);
```

Python:

```
ntp = np.array(ntp, dtype=np.int32)
nts = np.array(nts, dtype=np.int32)
ntv = np.array(ntv, dtype=np.int32)
(contn,contp) = voft.inte3d(c,ipv,nipv,ntp,nts,ntv,vertp,xnc,xns,
                           ync,yns,znc,zns)
```

Arguments:

<code>c</code>	IN	W_P	scalar representing the signed distance from the PLIC interface to the origin along the normal vector $\mathbf{n}$
<code>icontn</code>	OUT	I_P	number of original polyhedron vertices satisfying $\mathbf{n} \cdot \mathbf{x} + c \leq 0$
<code>icontp</code>	OUT	I_P	number of original polyhedron vertices satisfying $\mathbf{n} \cdot \mathbf{x} + c > 0$
<code>ipv</code>	INOUT	I_P	array of dimensions $(\mathbf{ns}, \mathbf{nv})$ storing the vertex indices defining each face boundary; updated for the truncated geometry
<code>nipv</code>	INOUT	I_P	array of length $\mathbf{ns}$ storing the number of vertices per face boundary; updated for the truncated geometry
<code>ntp</code>	INOUT	I_P	maximum vertex index in the connectivity list; updated after truncation
<code>nts</code>	INOUT	I_P	total number of face boundaries; updated after truncation
<code>ntv</code>	INOUT	I_P	total number of vertices; updated after truncation
<code>vertp</code>	INOUT	W_P	array of dimensions $(\mathbf{nv}, 3)$ storing the x, y, z -coordinates of each vertex; updated for the truncated geometry
<code>xnc, ync, znc</code>	IN	W_P	x, y, z -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase
<code>xns, yns, zns</code>	INOUT	W_P	arrays of length $\mathbf{ns}$ storing the x, y, z -components of the outward-pointing unit normal vector for each face boundary; updated for the truncated geometry

Note: The geometry is updated only when both `icontp` and `icontn` are positive. If `icontn` = 0, the entire polyhedron is retained and the geometry remains unchanged. If `icontp` = 0, the retained volume is zero, but the geometry arrays and size parameters also remain unchanged. The caller must therefore check these counters before using the returned geometry.

3.8 toolv3d – Volume computation for arbitrary polyhedra

The `toolv3d` routine computes the volume of an arbitrary polyhedron.

Fortran:

```
call toolv3d(ipv, nipv, nts, vertp, vol, xns, yns, zns)
```

C:

```
toolv3d(ipv, nipv, &nts, vertp, &vol, xns, yns, zns);
```

Python:

```
vol = voft.toolv3d(ipv, nipv, nts, vertp, xns, yns, zns)
```

Arguments:

ipv	IN	I_P	array of dimensions (ns,nv) storing the vertex indices defining each face boundary
nipv	IN	I_P	array of length ns storing the number of vertices per face boundary
nts	IN	I_P	total number of face boundaries
vertp	IN	W_P	array of dimensions (nv,3) storing the x,y,z-coordinates of each vertex
vol	OUT	W_P	scalar representing the signed volume of the polyhedron
xns, yns, zns	IN	W_P	arrays of length ns storing the x,y,z-components of the outward-pointing unit normal vector for each face boundary

Note: The routine computes an algebraic volume whose sign depends on the orientation of the face boundaries. Negative values are preserved and may be meaningful, for example when oriented donating polyhedra are used to represent fluid transport across cell faces. In such applications, the sign distinguishes inflow from outflow according to the adopted orientation convention.

3.9 intv3d – Half-space integration volume for arbitrary polyhedra

The intv3d routine computes the signed volume of the portion of a polyhedron lying in the half-space $\mathbf{n} \cdot \mathbf{x} + c > 0$. This corresponds to the liquid-side convention used by inte3d. The sign of the volume follows the boundary orientation convention described for toolv3d.

Fortran:

```
call intv3d(c, ipv, nipv, ntp, nts, vertp, vol, xnc, xns, &
           ync, yns, znc, zns)
```

C:

```
intv3d(&c, ipv, nipv, &ntp, &nts, vertp, &vol, &xnc, xns,
&ync, yns, &znc, zns);
```

Python:

```
vol = voft.intv3d(c, ipv, nipv, ntp, nts, vertp, xnc, xns, ync, yns, znc, zns)
```

Arguments:

c	IN	W_P	scalar representing the signed distance from the PLIC interface to the origin along the normal vector $\mathbf{n}$
---	----	-----	--

<code>ipv</code>	IN	I_P	array of dimensions (ns, nv) storing the vertex indices defining each face boundary
<code>nipv</code>	IN	I_P	array of length <code>ns</code> storing the number of vertices per face boundary
<code>ntp</code>	IN	I_P	maximum vertex index in the connectivity list
<code>nts</code>	IN	I_P	total number of face boundaries
<code>vertp</code>	INOUT	W_P	array of dimensions $(nv, 3)$ containing the vertex coordinates; rows beyond <code>ntp</code> are used to store additional intersection vertices during the calculation
<code>vol</code>	OUT	W_P	scalar representing the signed volume of the portion of the polyhedron lying within the half-space (liquid volume)
<code>xnc, ync, znc</code>	IN	W_P	x, y, z -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase
<code>xns, yns, zns</code>	IN	W_P	arrays of length <code>ns</code> storing the x, y, z -components of the outward-pointing unit normal vector for each face boundary

Note: Unlike `inte3d`, this routine does not return a complete representation of the truncated geometry. The original connectivity, size parameters, face normals, and vertex coordinates up to index `ntp` remain unchanged. However, additional intersection coordinates may be written to `vertp` beyond `ntp`, so the array must have sufficient capacity. In Python, `vertp` must be a writable, Fortran-contiguous NumPy array of the required data type.

3.10 `intc3d` – Arithmetic mean of intersection vertices for arbitrary polyhedra

The `tintc3d` routine computes the arithmetic mean of the intersection points generated where the polyhedron edges cross the plane $\mathbf{n} \cdot \mathbf{x} + c = 0$. The result is returned as the three-component vector `cen` in global coordinates. This provides a representative point for the cut feature, useful for initializing PLIC fitting procedures.

Fortran:

```
call intc3d(c, cen, iecen, ipv, nipv, ntp, nts, vertp, xnc, ync, znc)
```

C:

```
intc3d(&c, cen, &iecen, ipv, nipv, &ntp, &nts, vertp, &xnc, &ync, &znc);
```

Python:

```
(cen, iecen) = voft.intc3d(c, ipv, nipv, ntp, nts, vertp, xnc, ync, znc)
```

Arguments:

<code>c</code>	IN	W_P	scalar representing the signed distance from the PLIC interface to the origin along the normal vector $\mathbf{n}$
<code>cen</code>	OUT	W_P	vector representing the arithmetic mean of the intersection vertex coordinates
<code>iecen</code>	OUT	I_P	integer status flag: 1 if successful, 0 otherwise
<code>ipv</code>	IN	I_P	array of dimensions (ns, nv) storing the vertex indices defining each face boundary
<code>nipv</code>	IN	I_P	array of length <code>ns</code> storing the number of vertices per face boundary
<code>ntp</code>	IN	I_P	maximum vertex index in the connectivity list
<code>nts</code>	IN	I_P	total number of face boundaries
<code>vertp</code>	INOUT	W_P	array of dimensions $(nv, 3)$ containing the vertex coordinates; rows beyond <code>ntp</code> are used to store additional intersection vertices during the calculation
<code>xnc, ync, znc</code>	IN	W_P	x, y, z -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

Note: The routine preserves the original connectivity, size parameters, and vertex coordinates up to index `ntp`, but may write intersection coordinates to subsequent rows of `vertp`. The array must therefore have sufficient capacity and, in Python, must be writable and Fortran-contiguous.

Use `cen` only when `iecen`=1. When `iecen`=0, no intersection-point mean has been computed, and `cen` is set to zero.

The result is an arithmetic mean of intersection points, not an area-weighted centroid. For non-convex or disconnected sections, this point need not lie within the section. To compute the area centroid of the planar section, use `inte3d` to generate the truncated geometry, identify the face boundaries created by the cut, and compute the centroid by post-processing.

3.11 `inte3dface` – Half-space intersection for planar polygons

The `inte3dface` routine computes the intersection between an arbitrary oriented planar polygon (which can be either convex or non-convex) and a half-space. This routine is particularly useful in unsplit advection schemes based on the WATIA methodology, where accurate calculation of face intersections is critical. The retained region corresponds to the half-space $\mathbf{n} \cdot \mathbf{x} + c > 0$.

Fortran:

```
call inte3dface(c,icontn,icontp,ipv,ntp,ntv,vertp,xnc,ync,znc)
```

C:

```
inte3dface(&c,&icontn,&icontp,ipv,&ntp,&ntv,vertp,&xnc,&ync,&znc);
```

Python:

```
ntp = np.array(ntp, dtype=np.int32)
ntv = np.array(ntv, dtype=np.int32)
(icontn,icontp) = voft.inte3dface(c,ipv,ntp,ntv,vertp,xnc,ync,znc)
```

Arguments:

c	IN	W_P	scalar representing the signed distance from the PLIC interface to the origin along the normal vector $\mathbf{n}$
icontn	OUT	I_P	number of original polygon vertices satisfying $\mathbf{n} \cdot \mathbf{x} + c \leq 0$
icontp	OUT	I_P	number of original polygon vertices satisfying $\mathbf{n} \cdot \mathbf{x} + c > 0$
ipv	INOUT	I_P	array of length (nv) storing the vertex indices defining the arbitrary oriented planar polygon; updated for the truncated geometry
ntp	INOUT	I_P	maximum vertex index in the connectivity list; updated after truncation
ntv	INOUT	I_P	total number of vertices; updated after truncation
vertp	INOUT	W_P	array of dimensions (nv,3) storing the x, y, z -coordinates of each vertex; updated for the truncated geometry
xnc, ync, znc	IN	W_P	x, y, z -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

Note: The geometry is updated only when both `icontp` and `icontn` are positive. If `icontn` = 0, the entire polygon is retained and the geometry remains unchanged. If `icontp` = 0, the retained area is zero, but the geometry arrays and size parameters also remain unchanged. The caller must therefore check these counters before computing the retained area or exporting the resulting geometry.

3.12 areaface – Area of arbitrary planar polygons

The `areaface` routine computes the area of an arbitrary oriented planar polygon (either convex or non-convex). This routine is particularly useful for unsplit advection schemes based on the WATIA methodology, where face areas are required for flux calculations.

Fortran:

```
call areaface(a,ipv,ntv,vertp,xns,yns,zns)
```

C:

```
areaface(&a,ipv,&ntv,vertp,&xns,&yns,&zns);
```

Python:

```
a = voft.areaface(ipv,ntv,vertp,xns,yns,zns)
```

Arguments:

a	OUT	W_P	scalar representing the signed area of the arbitrary oriented planar polygon
ipv	IN	I_P	array of length (<i>nv</i>) storing the vertex indices defining the polygon perimeter
ntv	IN	I_P	total number of vertices
vertp	IN	W_P	array of dimensions (<i>nv</i> ,3) storing the <i>x,y,z</i> -coordinates of each vertex
xns, yns, zns	IN	W_P	<i>x,y,z</i> -components of the unit normal vector to the polygon plane

Note: The sign of *a* is determined by the vertex ordering relative to the supplied unit normal. Reversing either the vertex ordering or the normal direction reverses the sign; reversing both preserves it. The routine does not normalize the supplied normal vector. Signed areas may be used in fluid-flux calculations across cell faces. Their interpretation must be consistent with the velocity direction and the adopted face-orientation convention.

3.13 intpv3dpa – Intersection volume between paraboloid and arbitrary polyhedron

The `intpv3dpa` routine computes the volume of the intersection between a paraboloidal fluid region and an arbitrary polyhedron. This calculation employs an improved version of the approximate procedure based on recursive local grid refinement of the polyhedron proposed by López [14].

Fortran:

```
call intpv3dpa(cparab,ipv,nc,nipv,ntp,nts,ntv,vertp,vf,xns,yns,zns)
```

C:

```
intpv3dpa(cparab,ipv,&nc,nipv,&ntp,&nts,&ntv,vertp,&vf,xns,yns,zns);
```

Python:

```
vf=voft.intpv3dpa(cparab,ipv,nc,nipv,ntp,nts,ntv,vertp,xns,yns,zns)
```

Arguments:

<code>cparab</code>	IN	W_P	12-component array containing: the six paraboloid coefficients defining the surface geometry (see Eq. 3.1, indices 1-6); the three components of the unit vector along the local $\mathbf{f}$ -axis, expressed in global coordinates (indices 7-9); and the three coordinates of the local origin in global coordinates (indices 10-12). The component indices in the argument description follow Fortran's one-based indexing.
<code>ipv</code>	IN	I_P	array of dimensions $(\mathbf{ns}, \mathbf{nv})$ storing the vertex indices defining each face boundary of the polyhedron
<code>nc</code>	IN	I_P	scalar specifying the number of sub-cells along each coordinate axis of the superimposed Cartesian grid used for the refinement process
<code>nipv</code>	IN	I_P	array of length $\mathbf{ns}$ storing the number of vertices per face boundary
<code>ntp</code>	IN	I_P	maximum vertex index in the connectivity list
<code>nts</code>	IN	I_P	total number of face boundaries of the polyhedron
<code>ntv</code>	IN	I_P	total number of vertices in the polyhedron
<code>vertp</code>	IN	W_P	array of dimensions $(\mathbf{nv}, 3)$ storing the x, y, z -coordinates of each vertex
<code>vf</code>	OUT	W_P	scalar representing the volume of the portion of the polyhedron lying within the paraboloid (i.e., the liquid volume)
<code>xns, yns, zns</code>	IN	W_P	arrays of length $\mathbf{ns}$ storing the x, y, z -components of the outward-pointing unit normal vector for each face boundary

Note: Unlike PLIC methods based on planar interfaces, this routine uses a quadratic interface representation. The intersection volume is computed approximately, and its accuracy depends on the interface representation and the spatial refinement used in the calculation. Despite its name, `vf` is the computed intersection volume, not a volume fraction. For a cell of positive total volume `vt`, the corresponding liquid volume fraction is `vf/vt`. The subdivision parameter `nc` must be a positive integer.

3.14 box3d – Bounding box computation for point sets

The `box3d` routine computes the bounding box of a given set of points, defined as the smallest axis-aligned rectangular parallelepiped containing all points. It returns the minimum and maximum coordinates along each axis (x, y, z) .

Fortran:

```
call box3d(box, ntp, vertp)
```

C:

```
box3d(box,&ntp,vertp);
```

Python:

```
box = voft.box3d(ntp,vertp)
```

Arguments:

box	OUT	W_P	array of length (6) storing the bounding box coordinates. Indices are mapped as follows: $[x_{\min}, x_{\max}, y_{\min}, y_{\max}, z_{\min}, z_{\max}]$
ntp	IN	I_P	total number of points in the set
vertp	IN	W_P	array of dimensions $(nv, 3)$ storing the x, y, z -coordinates of each point

Note: The routine processes the first `ntp` rows of `vertp`, with $1 \leq ntp \leq nv$. It does not use connectivity information to identify active vertices. Consequently, unused vertices within this range are also included in the bounding-box calculation.

3.15 pvfit – Paraboloid fitting for polyhedral interface reconstruction

The `pvfit` routine fits a paraboloid to a set of `npol` polygons by constructing volumetric columns aligned with a local reference system (u, v, f) . The interface of this paraboloid, defined within the local reference system, is given by Eq. (3.1), where the coefficient vector is $\text{coef} = [c_1, c_2, c_3, c_4, c_5, c_6]$. Specifically, the routine projects the discrete polygons onto the local reference plane—defined by the orthonormal basis `vn0` and reference point `vp0`—thereby forming vertical columns bounded by the projected polygon edges. It then determines the paraboloid that best approximates the volume of these columns using a least-squares technique [4, 14].

Fortran:

```
call pvfit(coef,errfit,niv,npol,vn0,vp0,xn,xv,yn,yv,zn,zv)
```

C:

```
pvfit(coef,&errfit,niv,&npol,vn0,vp0,xn,xv,yn,yv,zn,zv);
```

Python:

```
(coef,errfit) = voft.pvfit(niv,npol,vn0,vp0,xn,xv,yn,yv,zn,zv)
```

Arguments:

coef	OUT	W_P	array of length (6) storing the coefficients $[c_1, c_2, c_3, c_4, c_5, c_6]$ of the fitted paraboloid in the local orthonormal basis
errfit	OUT	W_P	scalar representing the sum of squared volumetric residuals over the input polygons, evaluated using the fitted coefficients; this quantity is not normalized

<code>niv</code>	IN	I_P	array of length (<code>ns</code>) storing the number of vertices of each polygon
<code>npol</code>	IN	I_P	total number of polygons
<code>vn0</code>	IN	W_P	array of dimensions (3,3) whose rows contain the unit vectors of the local <i>f</i> -, <i>u</i> -, and <i>v</i> -axes, respectively, expressed in global coordinates
<code>vp0</code>	IN	W_P	array of length (3) storing the origin of the local reference system in global coordinates
<code>xn, yn, zn</code>	IN	W_P	arrays of length (<code>ns</code>) storing (<i>x</i> , <i>y</i> , <i>z</i>)-components of the unit-length normal vectors for each polygon
<code>xv, yv, zv</code>	IN	W_P	arrays of dimensions (<code>ns</code> , <code>nv</code>) storing the (<i>x</i> , <i>y</i> , <i>z</i>)-coordinates of every vertex of each polygon

Note: The polygon set must provide at least six independent volumetric constraints for the six fitted coefficients. The normal of each polygon must have a nonzero component along the local *f*-axis. Having six or more polygons alone does not guarantee a nonsingular fitting system.

3.16 `systra` – Rigid-body coordinate transformation

The `systra` routine transforms the global Cartesian coordinates (*x*, *y*, *z*) of a point into local coordinates (*f*, *u*, *v*). The transformation is defined by the orthonormal basis matrix `vn0`, whose rows contain the local *f*-, *u*-, and *v*-axis unit vectors expressed in global coordinates, and by `vp0`, the local origin expressed in global coordinates. The transformation is given by:

$$\begin{pmatrix} f \\ u \\ v \end{pmatrix} = \begin{pmatrix} vn0(1,1) & vn0(1,2) & vn0(1,3) \\ vn0(2,1) & vn0(2,2) & vn0(2,3) \\ vn0(3,1) & vn0(3,2) & vn0(3,3) \end{pmatrix} \begin{pmatrix} x - vp0(1) \\ y - vp0(2) \\ z - vp0(3) \end{pmatrix}$$

Fortran:

```
call systra(f,u,v,vn0,vp0,x,y,z)
```

C:

```
systra(&f,&u,&v,vn0,vp0,&x,&y,&z);
```

Python:

```
(f,u,v) = voft.systra(vn0,vp0,x,y,z)
```

Arguments:

<code>f, u, v</code>	OUT	W_P	scalars representing the coordinates of a point in the local reference system
<code>vn0</code>	IN	W_P	array of dimensions (3,3) whose rows contain the unit vectors of the local <i>f</i> -, <i>u</i> -, and <i>v</i> -axes, respectively, expressed in global coordinates

vp0	IN	W_P	array of length (3) storing the origin of the local reference system in global coordinates
x, y, z	IN	W_P	scalars representing the coordinates of a point in the global reference system

Note: The supplied basis vectors must be mutually orthogonal and of unit length. The routine neither normalizes nor orthogonalizes them.

3.17 cppol3d – Polyhedron deep copy

The `cppol3d` routine creates a deep copy of a polyhedron's geometric and topological structure. It duplicates the connectivity information, vertex coordinates, face normals, and plane distances, ensuring that the new polyhedron is an independent data structure from the original.

Fortran:

```
call cppol3d(cs,cs0,ipv,ipv0,nipv,nipv0,ntp,ntp0,nts,nts0, &
           ntv,ntv0,vertp,vertp0,xns,xns0,yns,yns0,zns,zns0)
```

C:

```
cppol3d(cs,cs0,ipv,ipv0,nipv,nipv0,&ntp,&ntp0,&nts,&nts0,
&ntv,&ntv0,vertp,vertp0,xns,xns0,yns,yns0,zns,zns0);
```

Python:

```
(cs,ipv,nipv,ntp,nts,ntv,vertp,xns,yns,zns) = voft.cppol3d(cs0,ipv0,
nipv0,ntp0,nts0,ntv0,vertp0,xns0,yns0,zns0)
```

Arguments:

cs	OUT	W_P	array of length <code>ns</code> storing the signed distance from the plane containing each face boundary of the copied polyhedron to the origin, along the outward-pointing normal vector
cs0	IN	W_P	array of length <code>ns</code> storing the signed distance from the plane containing each face boundary of the original polyhedron to the origin, along the outward-pointing normal vector
ipv	OUT	I_P	array of dimensions (<code>ns, nv</code>) storing the vertex indices defining the boundary of each face for the copied polyhedron
ipv0	IN	I_P	array of dimensions (<code>ns, nv</code>) storing the vertex indices defining the boundary of each face for the original polyhedron
nipv	OUT	I_P	array of length <code>ns</code> storing the number of vertices per face boundary for the copied polyhedron
nipv0	IN	I_P	array of length <code>ns</code> storing the number of vertices per face boundary for the original polyhedron

<code>ntp</code>	OUT	I_P	maximum vertex index in the connectivity list of the copied polyhedron
<code>ntp0</code>	IN	I_P	maximum vertex index in the connectivity list of the original polyhedron
<code>nts</code>	OUT	I_P	total number of face boundaries of the copied polyhedron
<code>nts0</code>	IN	I_P	total number of face boundaries of the original polyhedron
<code>ntv</code>	OUT	I_P	total number of vertices in the copied polyhedron
<code>ntv0</code>	IN	I_P	total number of vertices in the original polyhedron
<code>vertp</code>	OUT	W_P	array of dimensions $(nv, 3)$ storing the x, y, z -coordinates of each vertex of the copied polyhedron
<code>vertp0</code>	IN	W_P	array of dimensions $(nv, 3)$ storing the x, y, z -coordinates of each vertex of the original polyhedron
<code>xns, yns, zns</code>	OUT	W_P	arrays of length <code>ns</code> storing the x, y, z -components of the outward-pointing unit normal vector for each face of the copied polyhedron
<code>xns0, yns0, zns0</code>	IN	W_P	arrays of length <code>ns</code> storing the x, y, z -components of the outward-pointing unit normal vector for each face of the original polyhedron

Note: The source polyhedron is specified by the arguments with suffix 0. The routine copies the vertex coordinates up to `ntp0`, the face data up to `nts0`, and the connectivity entries up to `nipv0(i)` for each face. Entries outside these ranges are not defined by the routine. The plane constants `cs0` are copied without recomputation. Fortran and C callers must provide separate storage for the source and destination arrays.

3.18 dist3d – Point-to-convex-polygon Distance in 3D

The `dist3d` routine computes the shortest Euclidean distance from a point P in 3D space to a planar convex polygon defined by its vertex coordinates. The calculation accounts for whether the orthogonal projection of P onto the polygon's plane lies within or outside the polygon's boundary. If the projection is inside, the distance is the perpendicular distance to the plane; otherwise, it is the minimum distance to the polygon's edges or vertices.

Fortran:

```
call dist3d(d,n,x,y,z, xp, yp, zp)
```

C:

```
dist3d(&d,&n,x,y,z,&xp,&yp,&zp);
```

Python:

```
d = voft.dist3d(n,x,y,z,xp,yp,zp)
```

Arguments:

d	OUT	W_P	scalar representing the shortest Euclidean distance from point P to the polygon
n	IN	I_P	scalar representing the number of vertices defining the polygon
x, y, z	IN	W_P	arrays of length nv storing the coordinates of the polygon vertices
xp, yp, zp	IN	W_P	scalars storing the coordinates of point P

Note: The polygon must be planar, convex, and non-degenerate, with $3 \leq n \leq nv$. Its vertices must be supplied consecutively along the boundary, either clockwise or counterclockwise, without repeating the first vertex at the end. Only the first n entries of x , y , and z are used. The returned distance is non-negative.

3.19 initf3d – Volume fraction initialization via Cartesian shadow grid

The `initf3d` computes the volume fraction vf of a material body within an arbitrary polyhedral cell. The boundary of the material body is defined by a user-provided implicit function $\phi(x, y, z) \equiv \text{func3d}(x, y, z)$, with the material occupying the region $\phi(x, y, z) > 0$.

To approximate non-planar interfaces, the volume integral is evaluated using a composite numerical rule over a uniform Cartesian grid (shadow grid) superimposed on the polyhedral cell. This grid consists of $nc \times nc \times nc$ sub-cells, where nc is the resolution parameter. The integration scheme follows the methodology detailed in Refs. [7, 10].

Fortran:

```
call initf3d(func3d,ipv,nc,nipv,ntp,nts,ntv,tol,vertp,vf, &
            xns,yns,zns)
```

C:

```
initf3d(func3d,ipv,&nc,nipv,&ntp,&nts,&ntv,&tol,vertp,&vf,
xns,yns,zns);
```

Python:

```
vf = voft.initf3d(func3d,ipv,nc,nipv,ntp,nts,ntv,tol,vertp,xns,yns,
zns)
```

Arguments:

func3d	VOFTTOOLS_FUNC3D PROCEDURE		external procedure. User-defined implicit function defining the material interface. The routine samples this function to determine if points lie inside or outside the material
ipv	IN	I_P	array of dimensions (ns,nv) storing the vertex indices defining each face boundary of the polyhedron

<code>nc</code>	IN	I_P	scalar specifying the number of sub-cells along each coordinate axis of the superimposed Cartesian grid used for the refinement process in the numerical integration
<code>nipv</code>	IN	I_P	array of length <code>ns</code> storing the number of vertices per face boundary
<code>ntp</code>	IN	I_P	maximum vertex index in the connectivity list
<code>nts</code>	IN	I_P	total number of face boundaries of the polyhedron
<code>ntv</code>	IN	I_P	total number of vertices in the polyhedron
<code>tol</code>	IN	W_P	positive threshold parameter used in the preliminary vertex-based classification. Cartesian subdivision is performed when the vertex signs differ or when the minimum absolute value of <code>func3d</code> at the vertices is smaller than <code>tol*(xmax-xmin)</code> . This parameter is not an integration-error tolerance
<code>vertp</code>	IN	W_P	array of dimensions $(nv, 3)$ storing the x, y, z -coordinates of each vertex
<code>vf</code>	OUT	W_P	scalar representing the computed volume fraction of the material within the polyhedral cell. Value lies in $[0, 1]$
<code>xns, yns, zns</code>	IN	W_P	arrays of length <code>ns</code> storing the x, y, z -components of the outward-pointing unit normal vector for each face boundary

Note: Interfaces enclosed within the cell may be missed by the preliminary vertex-based classification. Increasing `nc` does not affect this classification. During subdivision, features that are not resolved by the subcell vertex samples may also remain undetected. The subdivision parameter `nc` must be a positive integer.

3.20 polout3d – VTK export for polyhedra

The `polout3d` routine exports the geometry and topological data of a polyhedron to an external file in VTK (visualization toolkit) format [5]. The generated files can be visualized using post-processing software such as ParaView [3]². The filename is automatically generated as `pol$ifile.vtk`, where `$ifile$` is the integer identifier provided by the user.

Fortran:

```
call polout3d(ifile,ipv,nipv,ntp,nts,vertp)
```

²The routine generates a legacy VTK file in ASCII format with dataset type `POLYDATA`, representing the polyhedrons polygonal surface. For polyhedra with non-convex polygonal faces, viewers like ParaView may require triangulation for correct rendering. Users are advised to apply a triangulation filter if visual artifacts occur.

C:

```
polout3d(&ifile, ipv, nipv, &ntp, &nts, vertp);
```

Python:

```
voft.polout3d(ifile, ipv, nipv, ntp, nts, vertp)
```

Arguments:

ifile	IN	I_P	scalar representing the integer identifier used to name the output file
ipv	IN	I_P	array of dimensions (ns,nv) storing the vertex indices defining each face boundary of the polyhedron
nipv	IN	I_P	array of length ns storing the number of vertices per face boundary
ntp	IN	I_P	maximum vertex index in the connectivity list
nts	IN	I_P	total number of face boundaries of the polyhedron
vertp	IN	W_P	array of dimensions (nv,3) storing the <i>x, y, z</i> -coordinates of each vertex

Note: The file is written to the current working directory. Its name consists of the prefix `pol`, the identifier `ifile` formatted as five digits with leading zeros, and the extension `.vtk`. For example, `ifile = 1` produces `pol00001.vtk`. Use identifiers in the range $0 \leq \text{ifile} \leq 99999$.

3.21 voftools_dim3d – Retrieve array dimensioning parameters for 3D polyhedra

The `voftools_dim3d` routine returns the array dimensioning parameters `ns` and `nv` compiled into the library. These values specify storage capacities, rather than the actual numbers of faces and vertices of a particular polyhedron.

Fortran:

```
call voftools_dim3d(ns, nv)
```

C:

```
voftools_dim3d(&ns, &nv);
```

Python:

```
(ns, nv) = voft.voftools_dim3d()
```

Arguments:

ns	OUT	I_P	compiled storage capacity for face entries
nv	OUT	I_P	compiled storage capacity for vertex entries

Note: Storage must accommodate any additional faces and vertices generated during geometric operations. This routine only reports the compiled dimensions; changing them requires rebuilding the library.

3.22 defpol2d – Initialize polygon type

The `defpol2d` routine initializes the geometry of a polygon by selecting one of the predefined shapes shown in Fig. 3.1, based on the integer input parameter `icelltype` (see Table 3.3).

Fortran:

```
call defpol2d(icelltype, ipv, ntp, ntv, vertp)
```

C:

```
defpol2d(&icelltype, ipv, &ntp, &ntv, vertp);
```

Python:

```
(ipv, ntp, ntv, vertp) = voft.defpol2d(icelltype)
```

<code>icelltype</code>	IN	I_P	cell geometry index
<code>ipv</code>	OUT	I_P	array of length <code>nv</code> storing the vertex indices defining the polygon
<code>ntp</code>	OUT	I_P	maximum vertex index in the connectivity list
<code>ntv</code>	OUT	I_P	total number of vertices
<code>vertp</code>	OUT	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex

Note: The `defpol2d` routine, used by the `test2d` program to select predefined cells, accepts `icelltype` values 1–6 for convex cells and 101–106 for non-convex cells. This selection does not restrict users to these geometries: user-defined polygonal cells can be supplied directly to the relevant VOFTools routines through their vertex coordinates and connectivity, subject to each routine’s requirements and the compiled array capacities. An invalid `icelltype` value causes `defpol2d` to print a diagnostic message and terminate the program through a Fortran `STOP` statement, including when called from C or Python.

3.23 enforv2d – PLIC interface location for 2D polygons

The `enforv2d` routine solves the two-dimensional VCE problem to locate the PLIC interface within a polygonal cell. Given a target liquid area v , it determines the position of the interface line such that the area of the liquid region within the cell matches the specified value.

The interface is defined by the equation $\mathbf{n} \cdot \mathbf{x} + c = 0$, where $\mathbf{n} = (x_{nc}, y_{nc})$ is the unit normal vector pointing towards the liquid phase, and c is a scalar parameter determining the interface position. The routine computes c such that:

$$\text{Area}(\{\mathbf{x} \in \Omega_{\text{cell}} | \mathbf{n} \cdot \mathbf{x} + c > 0\}) = v$$

where Ω_{cell} is the domain of the polygonal cell.

Fortran:

```
call enforv2d(c,ipv,ntp,ntv,v,vt,vertp,xnc,ync)
```

C:

```
enforv2d(&c,ipv,&ntp,&ntv,&v,&vt,vertp,&xnc,&ync);
```

Python:

```
c = voft.enforv2d(ipv,ntp,ntv,v,vt,vertp,xnc,ync)
```

Arguments:

c	OUT	W_P	scalar representing the signed distance from the PLIC interface to the origin along the unit normal vector $\mathbf{n}$
ipv	IN	I_P	array of length nv storing the vertex indices defining the polygon
ntp	IN	I_P	maximum vertex index in the connectivity list
ntv	IN	I_P	total number of vertices
v	IN	W_P	scalar representing the target liquid area within the cell
vt	IN	W_P	scalar representing the total area of the polygon
vertp	IN	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex
xnc, ync	IN	W_P	x, y -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

Note: This routine serves as the general solver for arbitrary 2D polygons. For axis-aligned rectangular cells, the more efficient analytical method in `enforv2dsz` should be preferred. The input polygon must have compact vertex numbering, with $ntp = ntv$. The total cell area vt must be positive, and the prescribed liquid area must satisfy $0 \leq v \leq vt$. The argument v is an area, not an area fraction; for a prescribed fraction α , use $v = \alpha vt$. The input polygon is not modified.

3.24 enforv2dsz – Efficient PLIC interface location for rectangular cells

The `enforv2dsz` routine solves the VCE problem specifically for axis-aligned rectangular cells, such as that defined in the `voftools_square` routine. It employs an efficient analytical solution proposed by Scardovelli and Zaleski [20], which provides a closed-form expression for the interface position c given the normal vector $\mathbf{n} = (xnc, ync)$ and target liquid area v .

Fortran:

```
call enforv2dsz(c,dx,dy,v,vertp,xnc,ync)
```

C:

```
enforv2dsz(&c,&dx,&dy,&v,vertp,&xnc,&ync);
```

Python:

```
v = np.array(v, dtype=np.float64)
c = vofte.enforv2dsz(dx,dy,v,vertp,xnc,ync)
```

Arguments:

c	OUT	W_P	scalar representing the signed distance from the PLIC interface to the origin along the unit normal vector $\mathbf{n}$
dx, dy	IN	W_P	scalars representing the cell dimensions along the x and y directions, respectively
v	INOUT	W_P	scalar representing the target liquid area within the cell; temporarily modified during the calculation and restored to its input value before returning
vertp	IN	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex
xnc, ync	IN	W_P	x, y -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

Note: The cell must be an axis-aligned rectangle whose four corner coordinates occupy the first four rows of `vertp`. The positive dimensions `dx` and `dy` must match its side lengths along the coordinate axes. The prescribed liquid area must satisfy $0 \leq v \leq dx dy$; for an area fraction α , use $v = \alpha dx dy$.

3.25 inte2d – Half-space truncation of arbitrary polygons

The `inte2d` routine performs geometric clipping of an arbitrary polygon against a half-space defined by the line $\mathbf{n} \cdot \mathbf{x} + c = 0$. It retains the portion of the polygon lying on the positive side of the line (i.e., where $\mathbf{n} \cdot \mathbf{x} + c > 0$, consistent with the liquid phase convention) and discards the portion on the negative side.

Fortran:

```
call inte2d(c,icontn,icontp,ipv,ntp,ntv,vertp,xnc,ync)
```

C:

```
inte2d(&c,&icontn,&icontp,ipv,&ntp,&ntv,vertp,&xnc,&ync);
```

Python:

```
ntp = np.array(ntp, dtype=np.int32)
ntv = np.array(ntv, dtype=np.int32)
(icontn, icontp) = voft.inte2d(c, ipv, ntp, ntv, vertp, xnc, ync)
```

Arguments:

c	IN	W_P	scalar representing the signed distance from the PLIC interface to the origin along the unit normal vector $\mathbf{n}$
icontn	OUT	I_P	number of original polygon vertices satisfying $\mathbf{n} \cdot \mathbf{x} + c \leq 0$
icontp	OUT	I_P	number of original polygon vertices satisfying $\mathbf{n} \cdot \mathbf{x} + c > 0$
ipv	INOUT	I_P	array of length nv storing the vertex indices defining the polygon; updated for the truncated geometry
ntp	INOUT	I_P	maximum vertex index in the connectivity list; updated after truncation
ntv	INOUT	I_P	total number of vertices; updated after truncation
vertp	INOUT	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex; updated for the truncated geometry
xnc, ync	IN	W_P	x, y -components of the unit-length vector $\mathbf{n}$ normal to the PLIC interface pointing towards the liquid phase

Note: The routine modifies `ipv`, `vertp`, `ntp`, and `ntv` in place only when both `icontn` and `icontp` are positive. If `icontn` = 0, the entire polygon is retained and the input geometry remains unchanged. If `icontp` = 0, the retained region is empty, but the input geometry also remains unchanged; the caller must handle this case explicitly. To preserve the original geometry when truncation occurs, make a copy before calling the routine, for example using `cppo12d`.

3.26 toolv2d – Area computation for arbitrary polygons

The `toolv2d` routine computes the signed area of an arbitrary polygon (convex or non-convex) defined by its vertices.

Fortran:

```
call toolv2d(ipv, ntv, vertp, vol)
```

C:

```
toolv2d(ipv, &ntv, vertp, &vol);
```

Python:

```
vol = voft.toolv2d(ipv, ntv, vertp)
```

Arguments:

ipv	IN	I_P	array of length nv storing the vertex indices defining the polygon
ntv	IN	I_P	total number of vertices
vertp	IN	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex
vol	OUT	W_P	scalar representing the signed area of the polygon

Note: The boundary traversal is defined by the first ntv entries of ipv . For a simple polygon, counterclockwise ordering gives a positive area and clockwise ordering gives a negative area. Reversing the traversal reverses the sign. For self-intersecting boundaries, the result is an algebraic signed area, which need not equal the geometric area of the enclosed regions. The routine preserves the sign, which may be meaningful in fluid-transport calculations according to the adopted orientation convention.

3.27 cppol2d – Polygon deep copy

The `cppol2d` routine creates a deep copy of a polygon's geometric and topological structure. It duplicates both the connectivity information (vertex indices) and the vertex coordinates, ensuring that the resulting polygon is an independent data structure from the original.

Fortran:

```
call cppol2d(ipv0, ipv, ntp0, ntp, ntv0, ntv, vertp0, vertp)
```

C:

```
cppol2d(ipv0, ipv, &ntp0, &ntp, &ntv0, &ntv, vertp0, vertp);
```

Python:

```
(ipv, ntp, ntv, vertp) = voft.cppol2d(ipv0, ntp0, ntv0, vertp0)
```

Arguments:

ipv	OUT	I_P	array of length nv storing the vertex indices defining the copied polygon
ipv0	IN	I_P	array of length nv storing the vertex indices defining the original polygon
ntp	OUT	I_P	maximum vertex index in the connectivity list of the copied polygon
ntp0	IN	I_P	maximum vertex index in the connectivity list of the original polygon
ntv	OUT	I_P	total number of vertices in the copied polygon
ntv0	IN	I_P	total number of vertices in the original polygon

vertp	OUT	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex of the copied polygon
vertp0	IN	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex of the original polygon

Note: In the calls shown above, the source polygon is specified by the arguments with suffix 0. The routine copies the first `ntv0` connectivity entries and the coordinates of the vertices referenced by those entries. Vertex numbering is preserved, and the values of `ntp0` and `ntv0` are copied to `ntp` and `ntv`. Other output array entries are not defined by the routine. Fortran and C callers must provide separate storage for the source and destination arrays.

3.28 dist2d – Point-to-segment distance

The `dist2d` routine computes the shortest Euclidean distance from a point P with coordinates (x_p, y_p) to the segment with endpoints $(x(1), y(1))$ and $(x(2), y(2))$. If the orthogonal projection of P onto the supporting line lies within the segment, the routine returns the perpendicular distance to that line; otherwise, it returns the distance to the nearest endpoint.

Fortran:

```
call dist2d(d,x,y,xp,yp)
```

C:

```
dist2d(&d,x,y,&xp,&yp);
```

Python:

```
d = voft.dist2d(x,y,xp,yp)
```

Arguments:

d	OUT	W_P	scalar representing the shortest Euclidean distance from point P to the segment
x, y	IN	W_P	arrays of length 2 storing the coordinates of the segment vertices
xp, yp	IN	W_P	scalars storing the coordinates of point P

Note: The segment endpoints must be distinct. The returned distance is non-negative and does not depend on the ordering of the endpoints. The routine does not explicitly handle zero-length segments.

3.29 initf2d – Area fraction initialization for 2D geometries via Cartesian shadow grid

The `initf2d` routine computes the fraction of the material area contained in a 2D cell. The boundary of the material is defined by a user-provided implicit function $\phi(x, y) \equiv \text{func2d}(x, y)$, with the material occupying the region $\phi(x, y) > 0$.

To approximate curved interfaces, the routine subdivides the cell's bounding box into a uniform Cartesian shadow grid with $nc \times nc$ subcells. Within each intersected subcell, interface intersections are estimated by linear interpolation of the sampled implicit-function values. The resulting material areas are accumulated and divided by the total cell area. Accuracy depends on the grid resolution and the interface geometry.

Fortran:

```
call initf2d(func2d,ipv,nc,ntp,ntv,tol,vertp,vf)
```

C:

```
initf2d(func2d,ipv,&nc,&ntp,&ntv,&tol,vertp,&vf);
```

Python:

```
vf = voft.initf2d(func2d,ipv,nc,ntp,ntv,tol,vertp)
```

Arguments:

func2d	VOFTOOLS_FUNC2D	PROCEDURE	external procedure. User-defined implicit function defining the material interface. The routine samples this function to determine if points lie inside or outside the material
ipv	IN	I_P	array of length nv storing the vertex indices defining the polygon
nc	IN	I_P	scalar specifying the number of sub-cells along each coordinate axis of the superimposed Cartesian grid used for the refinement process in the numerical integration
ntp	IN	I_P	maximum vertex index in the connectivity list
ntv	IN	I_P	total number of vertices in the polygon
tol	IN	W_P	positive threshold parameter used in the preliminary vertex-based classification. Cartesian subdivision is performed when the vertex signs differ or when the minimum absolute value of <code>func2d</code> at the vertices is smaller than $tol * (x_{max} - x_{min})$. This parameter is not an integration-error tolerance
vertp	IN	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex
vf	OUT	W_P	scalar representing the computed area fraction of the material within the polygonal cell. Value lies in $[0, 1]$

Note: Interfaces enclosed within the cell may be missed by the preliminary vertex-based classification. Increasing nc does not affect this classification. During subdivision, features that are not resolved by the subcell vertex samples may also remain undetected. The subdivision parameter nc must be a positive integer.

3.30 polout2d – Polygon geometry export in two-column format

The `polout2d` routine exports the geometric data of a polygon to an external file in a standard two-column ASCII format (e.g., x, y coordinates). This lightweight format is compatible with visualization tools such as `Gnuplot` [16]. The file is written to the current working directory. Its name consists of the prefix `pol`, the identifier `ifile` formatted as five digits with leading zeros, and the extension `.out`. For example, `ifile = 1` produces `pol00001.out`. Use identifiers in the range $0 \leq \text{ifile} \leq 99999$.

Fortran:

```
call polout2d(ifile,ipv,ntv,vertp)
```

C:

```
polout2d(&ifile,ipv,&ntv,vertp);
```

Python:

```
voft.polout2d(ifile,ipv,ntv,vertp)
```

Arguments:

<code>ifile</code>	IN	I_P	scalar representing the integer identifier used to name the output file (e.g., <code>pol00001.out</code>)
<code>ipv</code>	IN	I_P	array of length <code>nv</code> storing the vertex indices defining the polygon
<code>ntv</code>	IN	I_P	total number of vertices in the polygon
<code>vertp</code>	IN	W_P	array of dimensions $(nv, 2)$ storing the x, y -coordinates of each vertex

Note: The file contains three header lines beginning with `#`, followed by the vertex coordinates in the order specified by the first `ntv` entries of `ipv`. The first vertex is repeated at the end to close the polygon, giving `ntv+1` coordinate rows. Coordinates are written in two columns with six decimal places.

3.31 voftools_dim2d – Retrieve array dimensioning parameter for 2D polygons

The `voftools_dim2d` routine returns the vertex-array dimensioning parameter `nv` compiled into the library. This value specifies storage capacity, rather than the actual number of vertices of a particular polygon.

Fortran:

```
call voftools_dim2d(nv)
```

C:

```
voftools_dim2d(&nv);
```

Python:

```
nv = voft.voftools_dim2d()
```

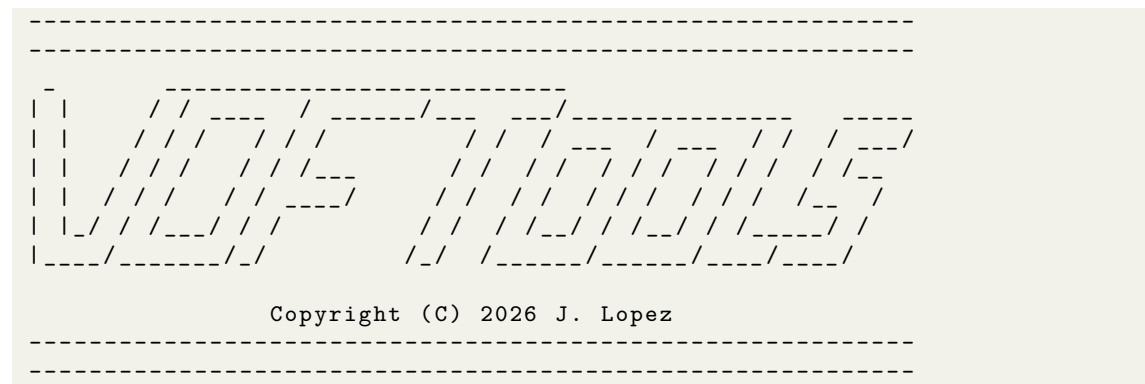
Argument:

nv OUT I_P compiled storage capacity for vertex entries

Note: Storage must accommodate any additional vertices generated during geometric operations. This routine only reports the compiled dimension; changing it requires rebuilding the library.

3.32 voftoolslogo – Print VOFTools logo

The voftoolslogo routine prints the VOFTools logo and copyright information on the screen.

**Fortran:**

```
call voftoolslogo
```

C:

```
voftoolslogo();
```

Python:

```
voft.voftoolslogo()
```


4 — Verification and validation

The `VOFTools` package includes a comprehensive suite of test programs designed to assess the accuracy, robustness, and performance of the implemented geometric operations. These tests are divided into two primary categories:

1. **PLIC operations:** focusing on piecewise linear interface reconstruction (Section 4.1), and
2. **PPIC operations:** focusing on piecewise paraboloidal interface reconstruction (Section 4.2).

4.1 PLIC operations

Test programs for both 2D and 3D PLIC operations are implemented in Fortran (`test2d.f90`, `test3d.f90`), C (`test2d.c`, `test3d.c`), and Python (`test2d.py`, `test3d.py`).

The configuration and execution of these test programs are detailed in Section 4.1.1. Subsequently, a runtime flow analysis along with quantitative accuracy results obtained from the tests are presented in Section 4.1.2.

4.1.1 Configuration and execution of test programs

The test programs require specific input data to define each test case. These inputs are categorized as follows:

- **Cell geometry:** The topological and geometric definition of the computational cell (vertices, connectivity, and face normals, for 3D cells).
- **Initial material distribution:** The material body geometry specified through an implicit function defining the initial material body, whose volume fraction within the cell is computed by the initialization procedure (`initf2d/initf3d`).
- **Target area/volume fraction:** The prescribed liquid fraction f . The target liquid area or volume supplied to the VCE solver is calculated as $v = f v_t$, where v_t is the total cell area or volume.
- **Interface normal vector:** The unit normal vector $\mathbf{n}$ to the reconstructed PLIC interface within the cell.

- **Distance evaluation point:** The coordinates of the point from which the Euclidean distance to the reconstructed PLIC interface is calculated (used in routines such as `dist2d/dist3d`).
- **Numerical parameters:** The Cartesian shadow-grid resolution `nc` and the threshold parameter `tol` used in the preliminary vertex-based classification of the initialization procedure.

For the Fortran and C test programs, input data are provided via the file `vofvardef`, included in the package distribution. Detailed information regarding this file is available in Chapter 2. These inputs are read by invoking the routine `vofvardef` located in `uservoftools.f90`. This routine can be invoked as follows:

Fortran:

```
call vofvardef(f, icelltype, ishape, nc, tol, xnc, xp, ync, yp, znc, &
              zp)
```

C:

```
vofvardef(&f, &icelltype, &ishape, &nc, &tol, &xnc, &xp, &ync, &yp, &znc,
          &zp);
```

Arguments:

<code>f</code>	OUT	W_P	target volume/area fraction
<code>icelltype</code>	OUT	I_P	cell geometry index (see Table 3.4)
<code>ishape</code>	OUT	I_P	material body shape index (see Table 4.1)
<code>nc</code>	OUT	I_P	sub-grid resolution along each axis
<code>tol</code>	OUT	W_P	positive threshold parameter used in the preliminary vertex-based classification of <code>initf2d/initf3d</code>
<code>xnc, ync, znc</code>	OUT	W_P	normal vector components $\mathbf{n}$
<code>xp, yp, zp</code>	OUT	W_P	Point P coordinates for distance calculation

Note: The indices for `icelltype` and `ishape` are defined in Tables 3.4 and 4.1, respectively. The implicit functions corresponding to each shape are defined in the file `uservoftools.f90`. The `vofvardef` input file must be available in the current working directory when running the Fortran or C test programs.

For the Python test programs `test2d.py` and `test3d.py`, input data are configured directly within the script files. This includes both the geometric parameters (cell geometry, target volume fraction, normal vector, etc.) and the user-defined implicit functions that define the material body shapes. This approach allows for immediate inspection and modification of test cases without relying on external configuration modules or compilation steps.

Table 4.1: Values of the `ishape` index used to denote the body shapes considered in the test programs.

<code>ishape</code> index	Body shape
<i>3D geometries</i>	
11	Sphere with radius 0.325 defined by the implicit function $f(x, y, z) = 0.325^2 - (x - 0.5)^2 - (y - 0.5)^2 - (z - 0.5)^2$
12	Torus with major radius $2/3$ and minor radius $1/3$ defined by the implicit function $f(x, y, z) = \left(\frac{1}{3}\right)^2 - \left\{\frac{2}{3} - [(x - 0.5)^2 + (z - 0.5)^2]^{1/2}\right\}^2 - (y - 0.5)^2$
13	Union between a sphere with radius 0.25 defined by the implicit function $\phi_1(x, y, z) = 0.25^2 - (x - 0.5)^2 - (y - 0.5)^2 - (z - 0.5)^2$ and a paraboloid defined by the implicit function $\phi_2(x, y, z) = z - 4x^2 - 4y^2 + 4(x + y) - 2.25$. The material region is defined by $\phi(x, y, z) = \max(\phi_1(x, y, z), \phi_2(x, y, z)) > 0$.
<i>2D geometries</i>	
1	Circle with radius 0.325 defined by the implicit function $f(x, y) = 0.325^2 - (x - 0.5)^2 - (y - 0.5)^2$
2	Ellipse with semi-major axis 0.5 and semi-minor axis 0.2 defined by the implicit function $f(x, y) = 1 - \left(\frac{x - 0.5}{0.5}\right)^2 - \left(\frac{y - 0.5}{0.2}\right)^2$

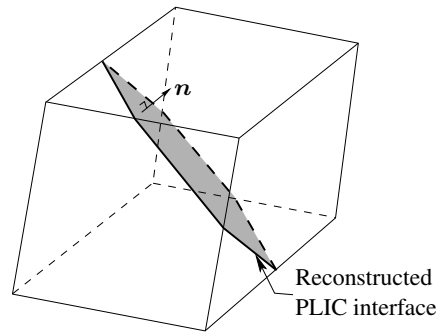


Figure 4.1: Positioning the interface by solving the VCE problem on a hexahedral cell.

Algorithmic workflow

The test programs execute a sequential workflow to validate each PLIC operation. The standard flow for both 2D and 3D cases is as follows:

1. **Cell Volume/Area Computation:** The total area/volume (v_t) of the computational cell is computed using `toolv3d` (3D) or `toolv2d` (2D). This value serves as the normalization reference for subsequent fraction calculations.
2. **Interface reconstruction (area/volume conservation enforcement):** A planar interface with a prescribed normal vector $\mathbf{n}$ is positioned within the cell such that it cuts off a target liquid area/volume fraction f (the solver receives $v = f v_t$). The position of this interface, defined by the constant c in the equation $\mathbf{n} \cdot \mathbf{x} + c = 0$, is determined by solving the VCE problem using `enforv3d` (3D) or `enforv2d` (2D) (see Fig. 4.1).
3. **Polyhedral/polygon clipping:** A working copy of the original cell geometry is truncated by the reconstructed interface plane/line. This is achieved by computing the intersection between the original cell and the half-space $\mathbf{n} \cdot \mathbf{x} + c > 0$ using `inte3d` (3D) or `inte2d` (2D) (see Fig. 4.2).
 - The routines `inte3d` and `inte2d` update the working geometry in place when truncation occurs. If `icontp` = 0, the retained region is empty and its volume or area must be set to zero explicitly, since the input geometry remains unchanged. Otherwise, its volume or area can be obtained using `toolv3d` or `toolv2d`.
 - In 3D, when explicit truncated geometry is not required, `intv3d` can compute the retained volume directly from the original cell. The routine `intc3d` returns the arithmetic mean of the intersection-vertex coordinates ($\times$ symbols in the left picture of Fig. 4.2), rather than an area or volume centroid.

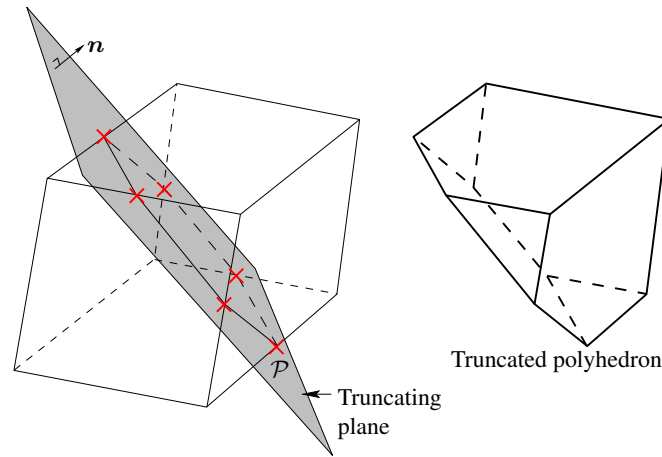


Figure 4.2: Truncation of the hexahedral cell by a plane containing the PLIC interface (the intersection vertices are marked with an $\times$ symbol in the left picture).

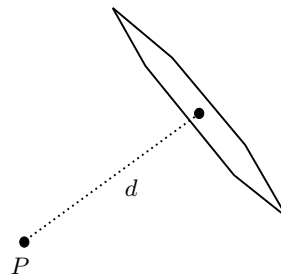


Figure 4.3: Computation of the distance from point P to the reconstructed PLIC interface.

4. **Distance calculation:** ¹ The Euclidean distance from a user-defined point P to the newly formed interface edge (in 2D) or face (in 3D) is computed using `dist2d` or `dist3d`, respectively, for convex cells (see Fig. 4.3).
5. **Initialization via implicit function:** To validate the initialization routines for complex interfaces, the volume/area fraction of a material body defined by an implicit function $\phi(x, y)$ in 2D and $\phi(x, y, z)$ in 3D (defined in `uservoofools.f90` for the Fortran/C examples and directly in the scripts for the Python examples) is computed within the cell using `initf3d` (3D) or `initf2d` (2D) (Fig. 4.4 shows a 3D example for a non-convex cell). This process involves superimposing a Cartesian shadow grid on the polyhedral/polygonal domain.

¹The distance calculation in these test programs is restricted to convex cells. For non-convex cells, the interface may consist of multiple segments or non-convex or disconnected polygonal regions, requiring additional geometric processing. The `dist2d` routine handles a single segment, whereas `dist3d` requires a planar convex polygon.

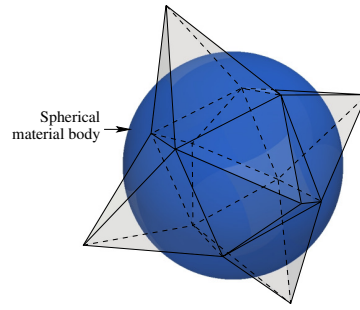


Figure 4.4: Initialization of the volume of a spherical material body contained inside a non-convex cell (example corresponding to `icelltype = 113` and `ishape = 11`).

In the following, several examples of the execution of the 2D and 3D test programs are presented.

Table 4.2: Input data for example 1.

<code>icelltype:</code>	1
<code>ishape:</code>	1
<code>f:</code>	0.5
<code>xnc, ync:</code>	0.0, -1.0
<code>xp, yp:</code>	0.0, 0.0
<code>nc:</code>	10
<code>tol:</code>	10.0

Example 1. Table 4.2 summarizes the input parameters considered for this 2D example. The execution of the 2D test program (`test2d_f`, `test2d_c`, or `python test2d.py`) yields the following results:

Total cell area, <code>vt</code> :	1.0
Solution of the VCE problem, <code>c</code> :	0.5
Distance from $\mathbf{x}_P = (x_p, y_p)$ to the interfacial segment, <code>d</code> :	0.5
Material area fraction in the cell, <code>vf</code> :	0.322155

Fig. 4.5 illustrates the results provided by the test program. By changing the position of point P to $(-1, 0)$, $(0.5, 0)$ and $(2, 0)$ the test program produces, respectively, the results shown in Figs. 4.6(a), 4.6(b) and 4.6(c). Note that the exact area fraction of the circular material body contained in the cell is

$$vf_{\text{exact}} = \pi 0.325^2.$$

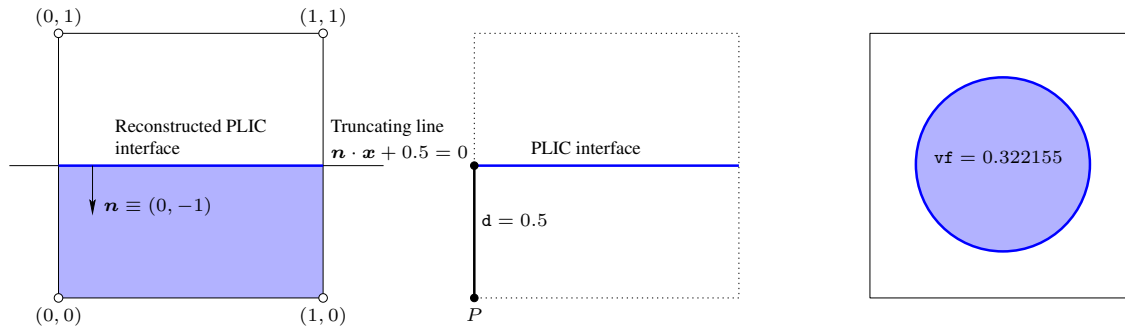


Figure 4.5: Illustration of the results produced by the 2D test program for the input data corresponding to those of Table 4.2.

Thus, the initialization error obtained using a division number $nc = 10$ is 9.7×10^{-3} . Increasing the division number to $nc = 20$, the material area fraction in the cell results as $vf = 0.329106$ producing an initialization error equal to 2.7×10^{-3} . Using the absolute error $E = |vf - vf_{\text{exact}}|$, these two resolutions give an observed convergence order of approximately $p = \log_2(E_{10}/E_{20}) = 1.83$, suggesting nearly second-order behavior for this example.

Table 4.3: Input data for example 2.

icelltype:	106
ishape:	1
f:	0.5
xnc, ync:	$1/\sqrt{2}, 1/\sqrt{2}$
xp, yp:	-
nc:	10
tol:	10.0

Example 2. This example demonstrates the PLIC reconstruction process for a non-convex polygonal cell. The distance calculation is omitted in this test because the reconstructed interface in a non-convex cell may consist of multiple segments, whereas `dist2d` computes the distance to a single segment. Table 4.3 shows the input data considered for this 2D example. The execution of the 2D test program (`test2d_f`, `test2d_c` or `python test2d.py`) produces the following results:

Area of the cell, vt:	0.79
Solution of the VCE problem, c:	$-1/\sqrt{2}$
Material area fraction in the cell, vf:	0.148487

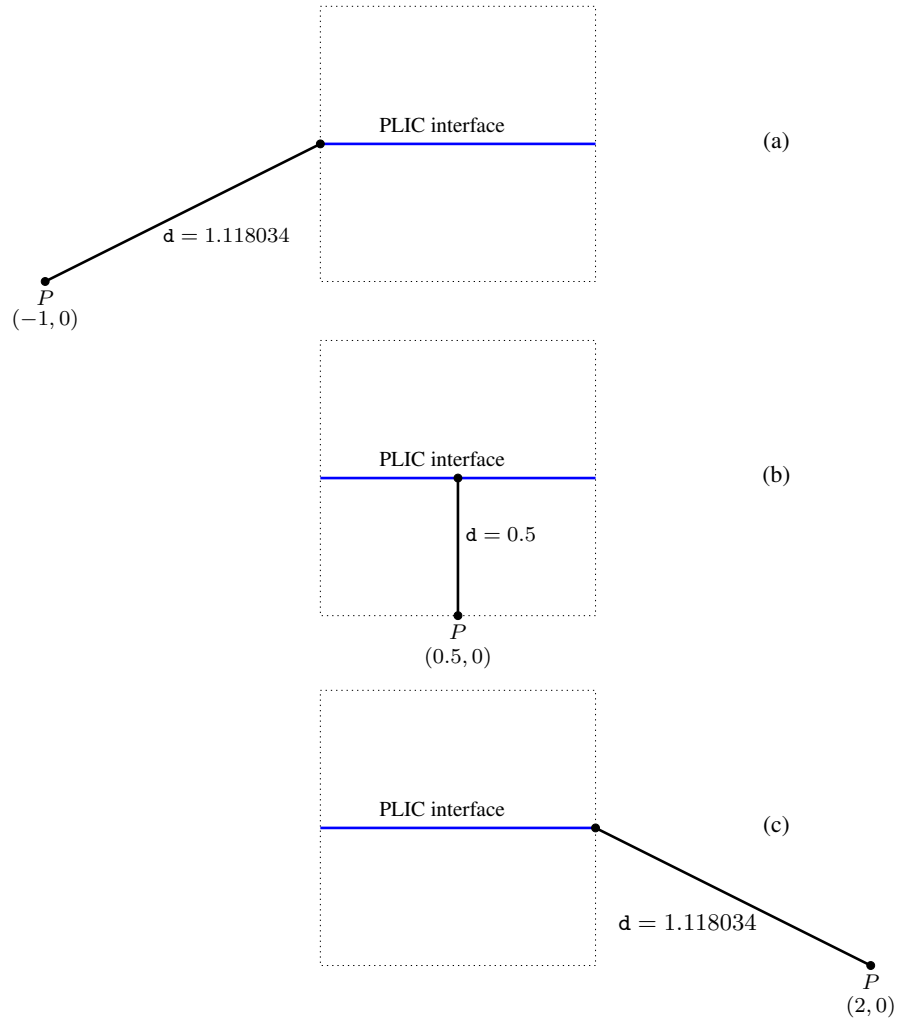


Figure 4.6: Distance d produced by the test program for the input data corresponding to those of Table 4.2 but changing the position of the point P to (a) $(-1, 0)$, (b) $(0.5, 0)$ and (c) $(2, 0)$.

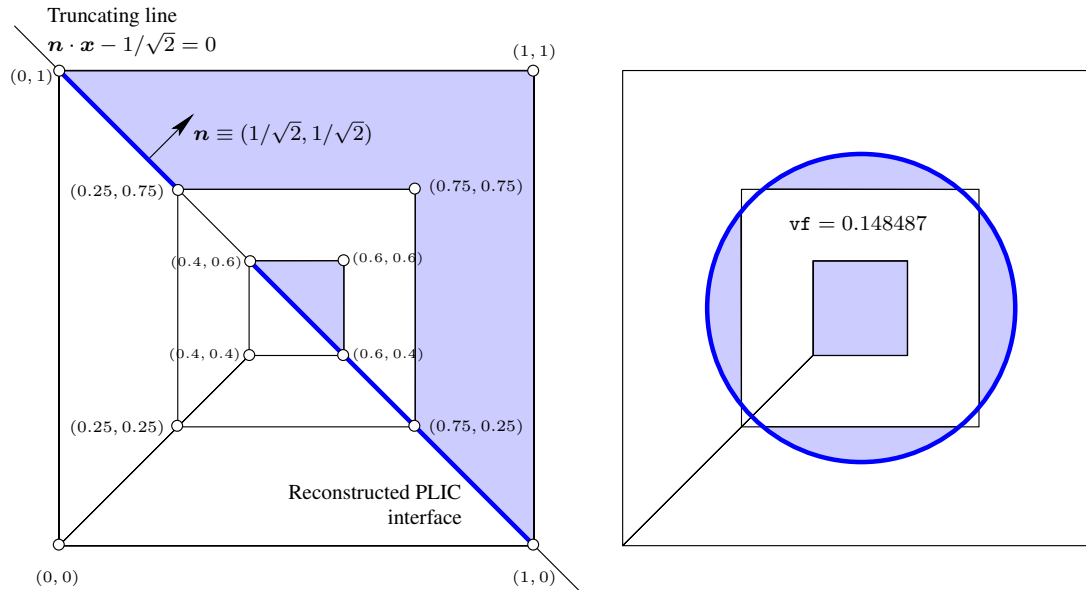


Figure 4.7: Illustration of the results produced by the 2D test program for the input data corresponding to those of Table 4.3.

Fig. 4.7 illustrates the results provided by the test program. The exact area fraction of the circular material body contained in the cell is, for this example,

$$\mathbf{vf}_{\text{exact}} = \frac{2 \times 0.325^2 (\theta - \sin \theta) + \frac{1}{5^2}}{1 - \frac{1}{2^2} + \frac{1}{5^2}},$$

where $\theta = 2 \arccos [1 / (2^2 \times 0.325)]$. Thus, the initialization error obtained using a division number $\mathbf{nc} = 10$ is 1.0×10^{-2} . Increasing the division number to $\mathbf{nc} = 20$, the material area fraction in the cell results as $\mathbf{vf} = 0.155460$ producing an initialization error equal to 3.0×10^{-3} . Using the absolute error $E = |\mathbf{vf} - \mathbf{vf}_{\text{exact}}|$, these two resolutions give an observed convergence order of approximately $p = \log_2(E_{10}/E_{20}) = 1.73$.

Table 4.4: Input data for example 3.

icelltype:	11
ishape:	11
f:	0.5
xnc, ync, znc:	0, -1, 0
xp, yp, zp:	0, 0, 0
nc:	10
tol:	10.0

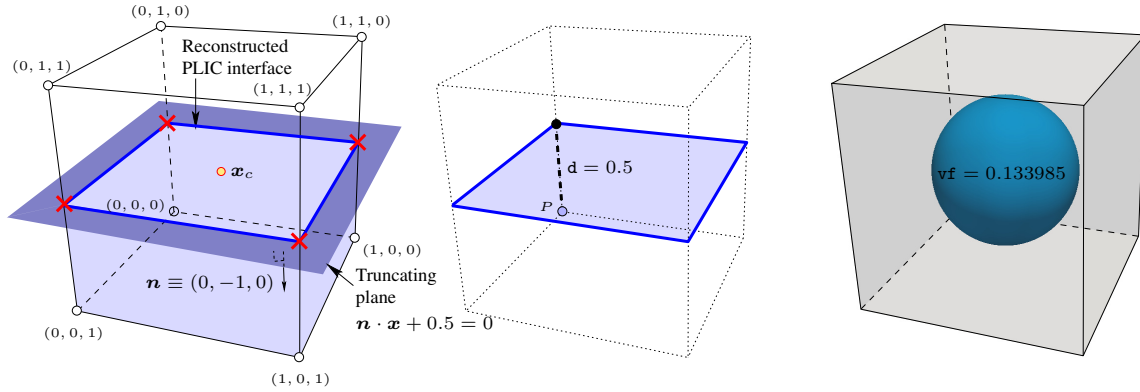


Figure 4.8: Illustration of the results produced by the 3D test program for the input data corresponding to those of Table 4.4.

Example 3. This example demonstrates the PLIC reconstruction process for a polyhedral cell. Table 4.4 summarizes the input data considered for this 3D example. The execution of the test program (`test3d_f`, `test3d_c`, or `python test3d.py`) yields the following results:

Volume of the cell, vt :	1.0
Solution of the VCE problem, c :	0.5
Truncated volume (<code>inte3d+toolv3d</code>), vol :	0.5
Truncated volume (<code>intv3d</code>), vol :	0.5
Geometric center $x_c = (x_c, y_c, z_c)$ of the set of intersection vertices:	(0.5, 0.5, 0.5)
Distance from $x_P = (x_p, y_p, z_p)$ to the interfacial polygon, d :	0.5
Material area fraction in the cell, vf :	0.133985

Fig. 4.8 illustrates the results provided by the test program. By changing the position of point P to $(0.5, 0, 0.5)$, $(-0.5, 0, 1.5)$ and $(1.5, 0, 0.5)$ the test program produces, respectively, the results shown in Figs. 4.9(a), 4.9(b) and 4.9(c). The exact volume fraction of the spherical material body contained in the cell is

$$vf_{\text{exact}} = \frac{4}{3}\pi 0.325^3.$$

The absolute initialization error is approximately 9.8×10^{-3} for $nc = 10$. Increasing the resolution to $nc = 20$ gives $vf = 0.141256$, with an absolute error of approximately 2.5×10^{-3} . These two resolutions give an observed convergence order of approximately $p = \log_2(E_{10}/E_{20}) = 1.95$, consistent with nearly second-order behavior for this example.

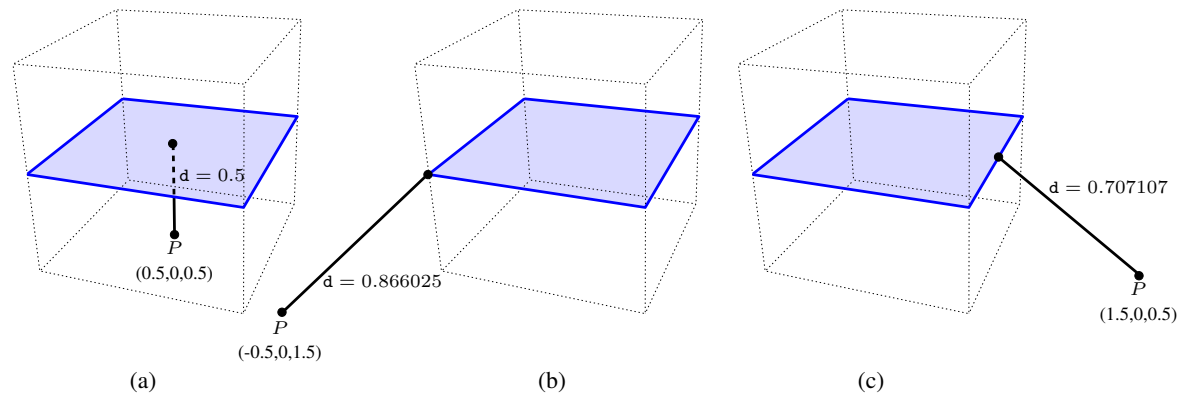


Figure 4.9: Distance d produced by the test program for the input data corresponding to those of Table 4.4 but changing the position of the point P to (a) $(0.5, 0, 0.5)$, (b) $(-0.5, 0, 1.5)$ and (c) $(1.5, 0, 0.5)$.

Table 4.5: Input data for example 4.

icelltype:	113
ishape:	12
f:	0.5
xnc, ync, znc:	0, -1, 0
xp, yp, zp:	-
nc:	10
tol:	10.0

Example 4. This example demonstrates the PLIC reconstruction process for a non-convex polyhedral cell. Table 4.5 shows the input data considered for this 3D example. The execution of the 3D test program (`test3d_f` or `test3d_c`) produces the following results:

Volume of the cell, vt:	3.0
Solution of the VCE problem, c:	0.5
Truncated volume (<code>inte3d+toolv3d</code>), vol:	1.5
Truncated volume (<code>intv3d</code>), vol:	1.5
Geometric center $x_c = (x_c, y_c, z_c)$ of the set of intersection vertices:	(0.5, 0.5, 0.0)
Material volume fraction in the cell, vf:	0.321329

Fig. 4.10 illustrates the geometric configuration and results provided by the test program. A reference volume fraction is estimated using Richardson extrapolation from the solutions obtained with $nc = 512$ and $nc = 1024$, assuming a leading

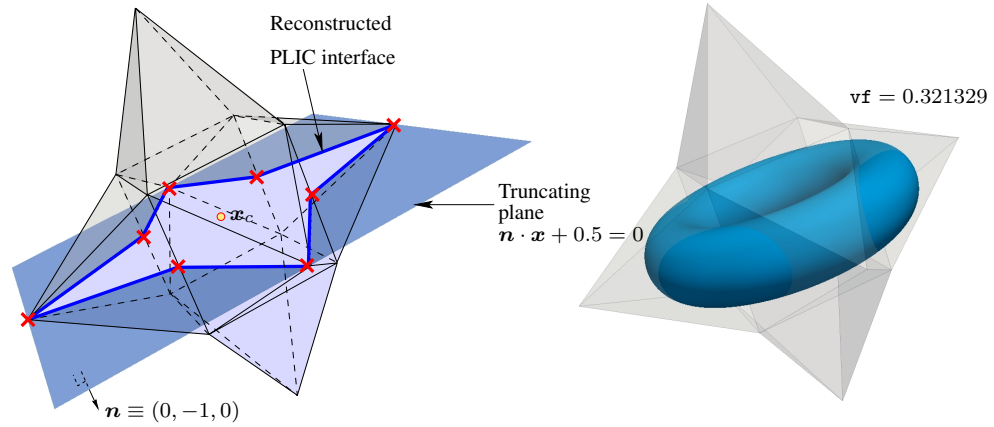


Figure 4.10: Illustration of the results produced by the test program for the input data corresponding to those of Table 4.5.

second-order discretization error.

$$\mathbf{vf}_{\text{ext}} = \frac{4}{3}\mathbf{vf}_{1024} - \frac{1}{3}\mathbf{vf}_{512},$$

The initialization error $|\mathbf{vf} - \mathbf{vf}_{\text{ext}}|$ was evaluated for coarser resolutions:

- For $\mathbf{nc} = 10$, the error is 6.9×10^{-2} .
- For $\mathbf{nc} = 20$, the computed fraction is $v_f = 0.372632$, yielding an error of 1.8×10^{-2} .

The convergence order is estimated as:

$$p \approx \log_2 \left(\frac{6.9 \times 10^{-2}}{1.8 \times 10^{-2}} \right) \approx 1.94$$

The observed error reduction is consistent with approximately second-order behavior over these two resolutions, using the extrapolated solution as a numerical reference.

4.1.2 Runtime workflow analysis and accuracy validation of PLIC operations

This section presents a detailed runtime workflow analysis alongside quantitative accuracy results for various PLIC routines. Comprehensive assessments of these methods are provided in References [6–12, 15].

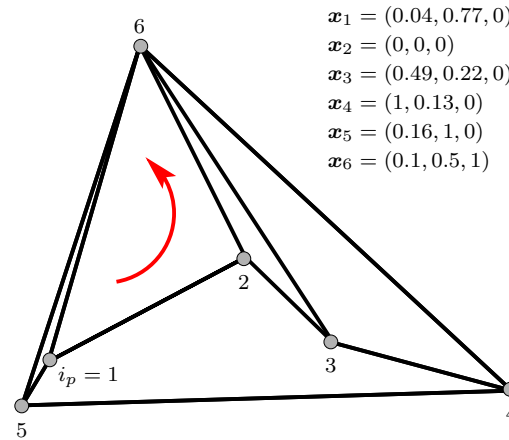


Figure 4.11: Position coordinates of the vertices of the non-convex pentagonal pyramid shown in Fig. 3.2.

Table 4.6: Array ipv0 of vertex number, i_p , assigned to every vertex index i of face boundary j of the polyhedron of Fig. 4.11. Each table entry is $\text{ipv0}(j, i)$.

Vertex index i	Face boundary j					
	1	2	3	4	5	6
1	1	1	2	3	4	5
2	5	2	3	4	5	1
3	4	6	6	6	6	6
4	3	—	—	—	—	—
5	2	—	—	—	—	—

Volume truncation

Firstly, the non-convex pentagonal pyramid shown in Fig. 3.2, which is defined by the supplied routine `voftools_ncpentapy` (the position coordinates of its vertices are shown in Fig. 4.11), is considered. Table 4.6 presents the arrangement of the vertices of this polyhedron. Fig. 4.12 shows the truncation operation, performed by calling the routine `inte3d` (Section 3.7), over the considered polyhedron and the plane $\mathcal{P}$ with interface orientation $\mathbf{n}$ given by $x_{nc} = 0$, $y_{nc} = -1$ and $z_{nc} = 0$, and position given by $c = 0.18$. Internally, an auxiliary array $\mathbf{ia}$ of length $\mathbf{nv}$ is constructed to store the information of the position of each vertex with respect to the plane $\mathcal{P}$, where a value equal to 1 indicates that the vertex is located in the half-space defined by $\mathbf{n} \cdot \mathbf{x} + c > 0$ and a value equal to 0 indicates that the vertex is located in the half-space defined by $\mathbf{n} \cdot \mathbf{x} + c \leq 0$. The vertices with $\mathbf{ia}$ array values equal to 1 and 0 are represented in Fig. 4.12 with black and white circles, respectively. As described

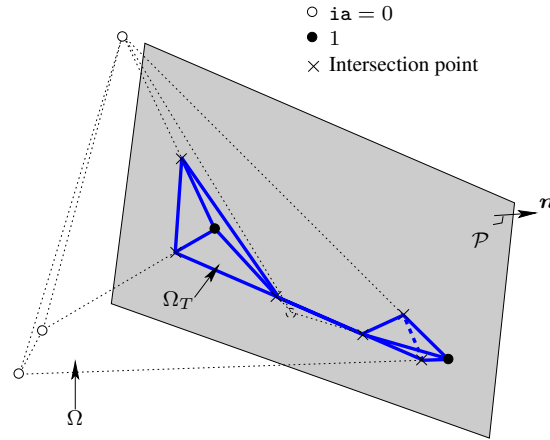


Figure 4.12: Truncation of the polyhedron Ω of Fig. 4.11 (dotted lines) by a plane $\mathcal{P}$ defined by $x_{nc} = 0$, $y_{nc} = -1$, $z_{nc} = 0$ and $c = 0.18$ (highlighted in gray). The truncated polyhedron Ω_T is represented with blue thick lines.

below, the resulting truncated region Ω_T (thick lines in Fig. 4.12) will be defined by the vertices where the ia value is equal to 1 and the points of intersection between $\mathcal{P}$ and the edges of Ω ($\times$ symbols in Fig. 4.12). For example, one of these edges is defined by the two adjacent vertices, $\mathbf{x}_5$ and $\mathbf{x}_4$, and the position vector of the intersection point results as $\mathbf{x}_7 = \mathbf{x}_4 - \frac{\phi_4}{\phi_5 - \phi_4} (\mathbf{x}_5 - \mathbf{x}_4)$, where $\phi_i = \mathbf{n} \cdot \mathbf{x}_i + c$. The truncation procedure begins by discarding the polyhedron face boundaries in which all the vertices have null value for the array ia (discarded face boundary 6 in Fig. 4.13) by setting the corresponding value of array $nipv1$ to 0. Next, the new array $ipv1$ of ordered vertex indices for every truncated face boundary (face boundaries 1, 2, 3, 4 and 5 in Fig. 4.14) and every new on- $\mathcal{P}$ face boundary of the truncated polyhedron (face boundary 7 in Fig. 4.15) is constructed following, respectively, the clipping and capping procedures described in [10]. Fig. 4.15 illustrates the sequence of the on- $\mathcal{P}$ face boundary construction. Intersection vertices are represented with $\times$, truncated faces are gray filled and the polygonal boundary of the constructed on- $\mathcal{P}$ face is highlighted at the end of the sequence in blue thick lines. The edge of a truncated face boundary containing a “key vertex” [10] and the edge of that face boundary through which the edges of the on- $\mathcal{P}$ face boundaries are sequentially constructed are also highlighted in each picture as $\circ \rightarrow \bullet$ and $\bullet \rightarrow \circ$, respectively. In this example, the resulting on-plane face boundary is a weakly simple polygonal chain. Edges 7-10 and 9-8 partially overlap and traverse their common portion in opposite directions, allowing the two disjoint coplanar regions (highlighted in blue in Fig. 4.16) to be represented by a single closed chain as $ipv1(7, i) = (7, 10, 11, 9, 8, 12)$ for $i = 1$ to $nipv1(7) = 6$ (Fig. 4.16 shows this arrangement). Table 4.7 shows the vertex arrange-

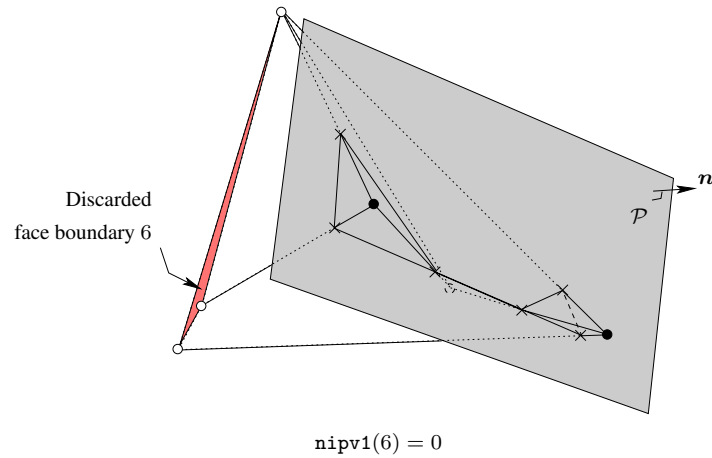


Figure 4.13: Face discarding.

Table 4.7: Array ipv1 of the index number, i_p , assigned to every vertex index i of face boundary j of the polyhedron, Ω_T , resulting from the truncation operation of Fig. 4.12. Each table entry is $\text{ipv1}(j, i)$.

Vertex index i	Face boundary j						
	1	2	3	4	5	6	7
1	7	10	2	8	4	—	7
2	4	2	9	4	7	—	10
3	8	11	11	12	12	—	11
4	9	—	—	—	—	—	9
5	2	—	—	—	—	—	8
6	10	—	—	—	—	—	12

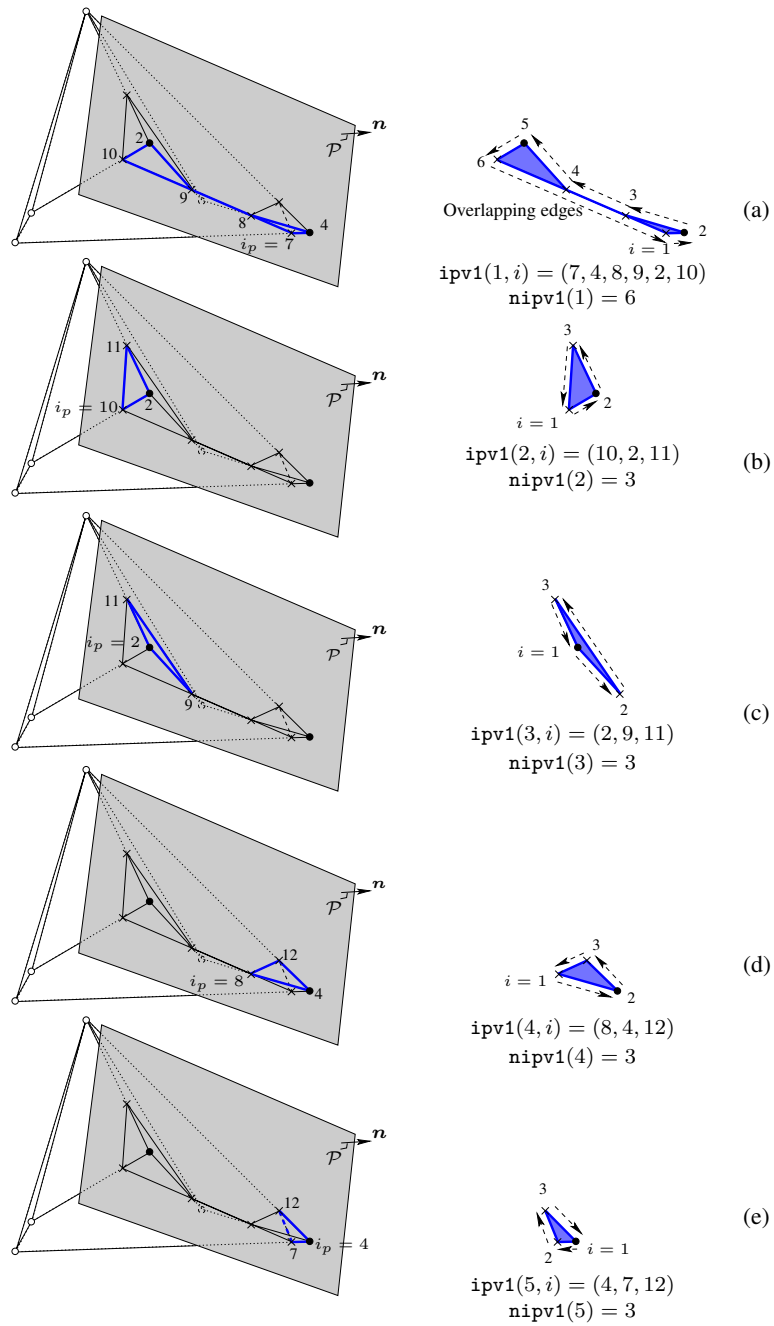


Figure 4.14: Clipping procedure for the face boundaries (a) 1, (b) 2, (c) 3, (d) 4 and (e) 5 of Fig. 4.12. Polygon truncation on the left and truncated polygons, filled with blue (arrows indicate vertex order), on the right.

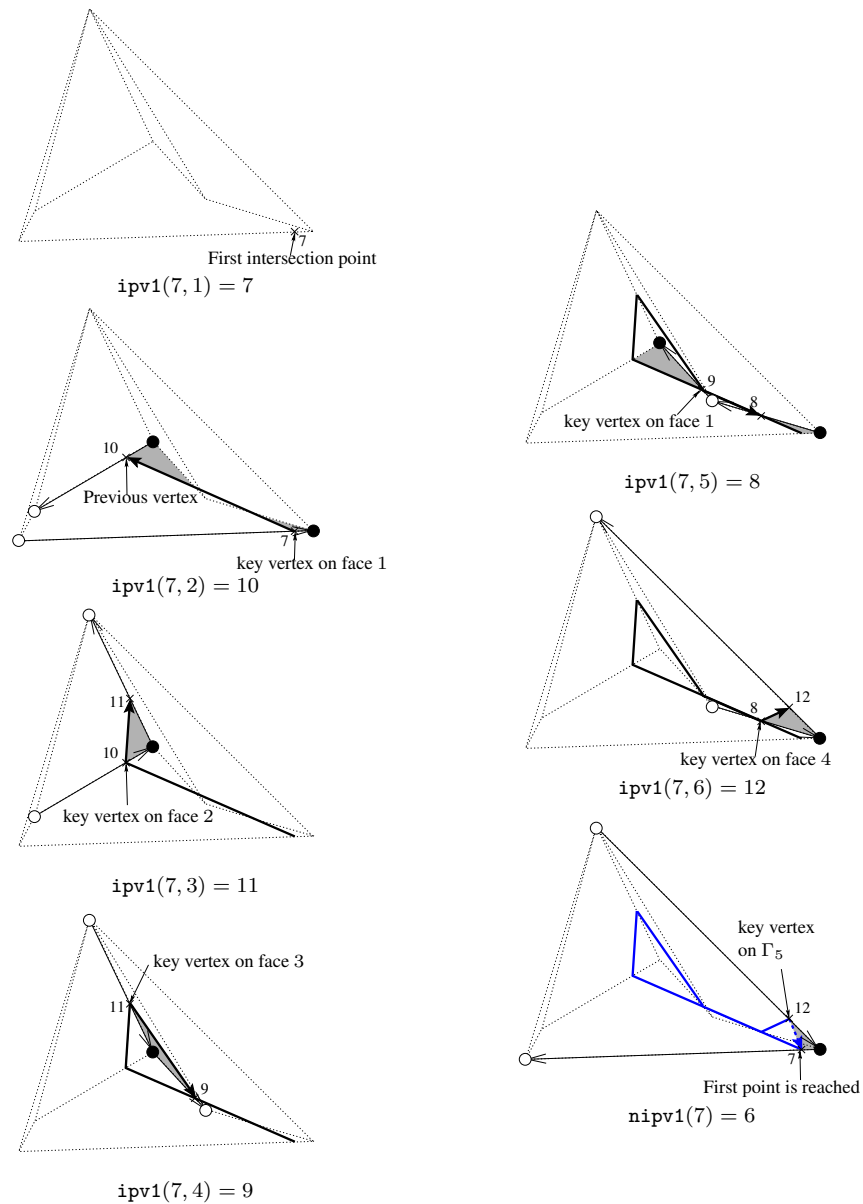


Figure 4.15: Sequence (from top to bottom and from left to right) of the capping procedure applied for the construction of the on- $\mathcal{P}$ face boundary (thick lines) in the example of Fig. 4.12.

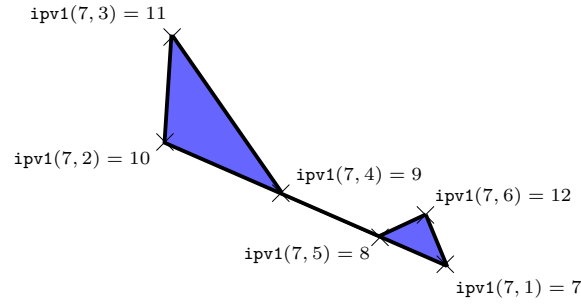


Figure 4.16: Vertex arrangement of the on- $\mathcal{P}$ face boundary in the example of Fig. 4.12.

ment of the polyhedron, Ω_T , resulting from the truncation operation of Fig. 4.12.

A case in which the application of the capping procedure produces multiple new on- $\mathcal{P}$ face boundaries (four in total) can be seen in Fig. 4.17.

Volume conservation enforcement in PLIC reconstruction

Figs. 4.18 and 4.19 show the sequence of application of the VCE operation, performed by calling the routine `enforv3d`, over the cell of Fig. 4.11 for a case with PLIC interface orientation $\mathbf{n}$ given by $x_{nc} = 0$, $y_{nc} = -1$ and $z_{nc} = 0$, and ratio between the reference material and cell volumes (volume fraction f) $v/vt = 0.55$ (a sketch of the flow diagram of the VCE operation is also included in the figures to highlight the applied steps). The sequence for this case occurs as follows.²

1. For this geometry and normal direction, the initial bounds are $c = 0$ and $c = 1$, corresponding to empty and full cells, respectively. Linear interpolation (see Fig. 4.18(a)) between these bounds therefore gives:

$$ct1 = \frac{v}{vt} = 0.55.$$

2. The section of the function $v(c)$ that satisfies the condition $c_{min} \leq ct1 < c_{max}$ is searched (Fig. 4.18(b)). The identified section is delimited by the tentative bracket values $c_{min} = 0.5$ and $c_{max} = 0.77$ (these values correspond to the constant of the planar interfaces passing through vertices 6 (thus, $c_{min} = -x_{nc} \times vertp(6, 1) - y_{nc} \times vertp(6, 2) - z_{nc} \times vertp(6, 3)$) and 1 (thus, $c_{max} = -x_{nc} \times$

²For clarity in the explanation of this example, it has been omitted the activation, when appropriate to achieve a better computational efficiency, of the solution of

$$v(c) - (vt - v) = 0,$$

with interface normal $-\mathbf{n}$, instead of the solution of $v(c) - v = 0$ with interface normal $\mathbf{n}$ (see Reference [8] for the details).

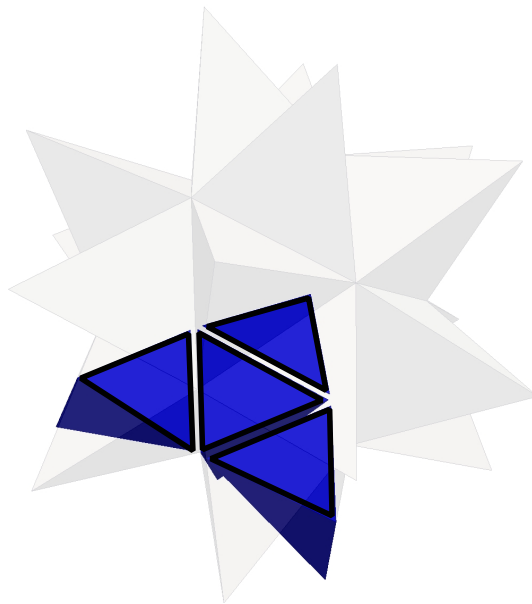


Figure 4.17: Example of the truncation between the stellated icosahedron of Fig. 3.2 and a half-space that produces four new on- $\mathcal{P}$ face boundaries (black thick lines). The truncated regions are highlighted in blue.

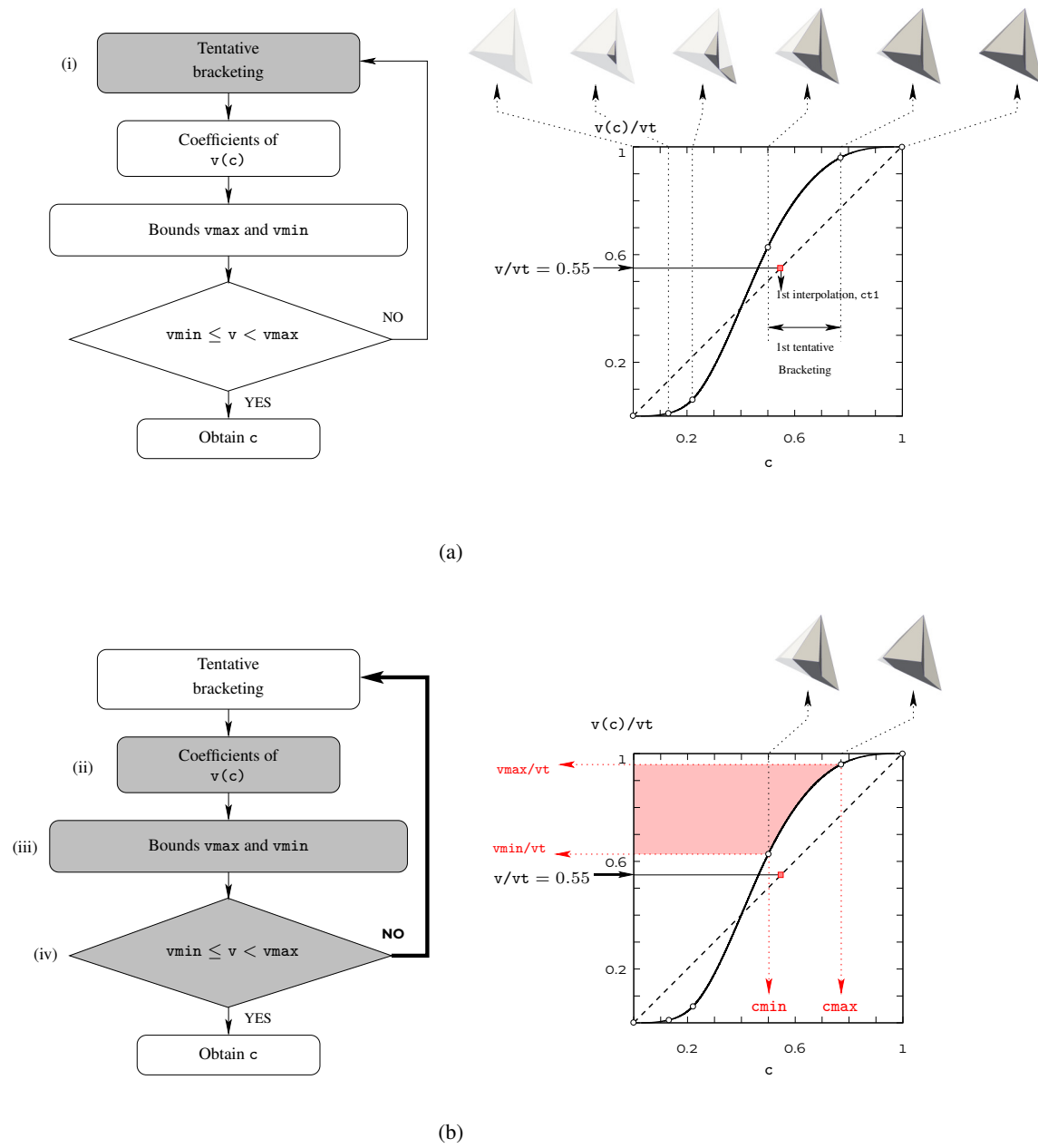
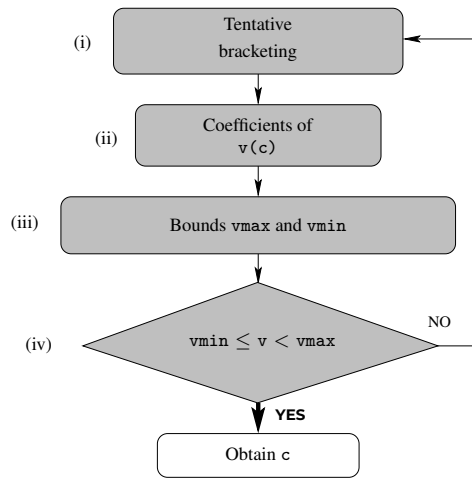
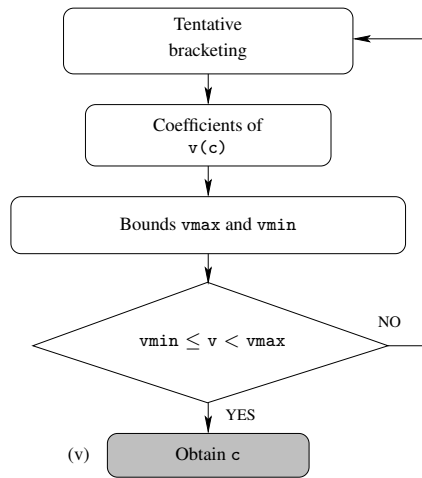
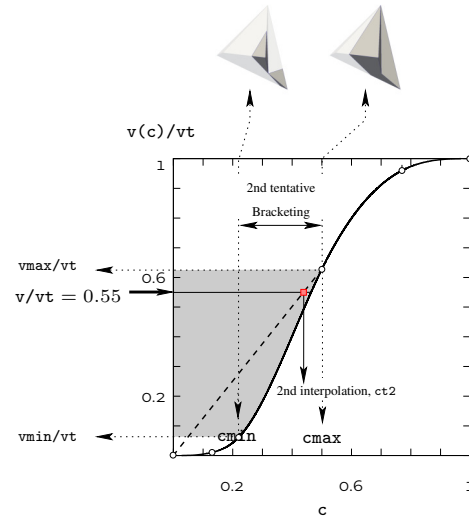


Figure 4.18: Sequence of application of the VCE operation in a PLIC reconstruction within the cell of Fig. 4.11 for a case with $x_{nc} = 0$, $y_{nc} = -1$, $z_{nc} = 0$ and $f = v/vt = 0.55$. (a) First tentative value of the solution and identification of the bracketing interval. (b) Computation of the coefficients of the analytical function $v(c)$ valid inside the tentative bracketing interval, from which the bounds v_{max} and v_{min} are obtained, and check if the condition $v_{min} \leq v < v_{max}$ is or not satisfied.



(a)



(b)

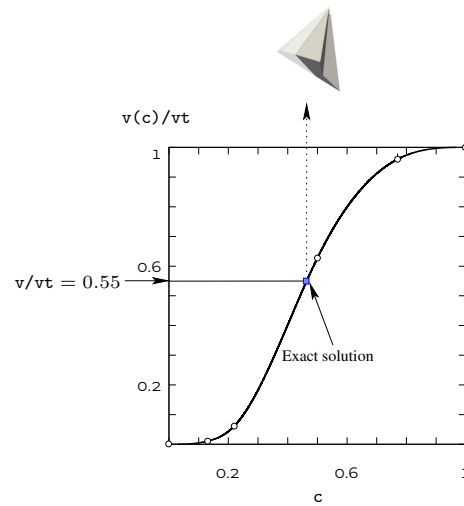


Figure 4.19: Continuation of the sequence of Fig. 4.18. (a) Steps (i) to (iii) are repeated and the volume condition is checked again. (b) The volume condition is satisfied and the final solution is analytically obtained.

$\text{vertp}(1,1) - \text{ync} \times \text{vertp}(1,2) - \text{znc} \times \text{vertp}(1,3))$ and the corresponding coefficients of the function $v(c)/vt$ are obtained from the analytical relations given in [10], resulting for this section as (the coefficients are displayed to five decimal places)

$$\frac{v(c)}{vt} = 1.36248c^3 - 5.41244c^2 + 6.43269c - 1.40609.$$

From the above expression, the bounds of the first tentative bracket are

$$\frac{v_{\max}}{vt} = 0.96006$$

and

$$\frac{v_{\min}}{vt} = 0.62745.$$

Note that the tentative bracket determined by these bounds does not contain the value $v/vt = 0.55$ and a new tentative interpolation bracketing must be carried out.

3. A second tentative value, $ct2$, is obtained by using the linear interpolation shown in Fig. 4.19(a) for which the upper limit of the interpolation line has been moved to the point on the curve $v(c)/vt$ corresponding to $c = 0.5$. In this way, the new interpolated tentative value results as

$$ct2 = 0.43828.$$

The section of the function $v(c)$ that satisfies the condition $c_{\min} \leq ct2 < c_{\max}$ is now delimited by the planar interfaces passing through vertices 6 (thus, $c_{\max} = -xnc \times \text{vertp}(6,1) - ync \times \text{vertp}(6,2) - znc \times \text{vertp}(6,3) = 0.5$) and 3 (thus, $c_{\min} = -xnc \times \text{vertp}(3,1) - ync \times \text{vertp}(3,2) - znc \times \text{vertp}(3,3) = 0.22$) and the corresponding analytical expression of $v(c)/vt$ for this section results from [10] as

$$\frac{v(c)}{vt} = -12.29024c^3 + 15.06660c^2 - 3.80685c + 0.30050.$$

The bounds of the second tentative bracket are

$$\frac{v_{\max}}{vt} = 0.62745$$

and

$$\frac{v_{\min}}{vt} = 0.06135.$$

This tentative bracket includes the value $v/vt = 0.55$.

4. The final solution is analytically computed (Fig. 4.19(b)) as indicated in [6], resulting

$$c = 0.46394.$$



Figure 4.20: Liquid volume regions (blue color) obtained from the intersection between the VOFTools logo cell (routine `voftools_logo`; see Fig. 3.2) and half-spaces defined by planes that contain the PLIC interfaces with normal vector $\mathbf{n} = (0, -1, 0)$ and different liquid volume fractions v/v_t (the solutions of the VCE problem are included in parenthesis). The original cell is depicted on each picture using partial transparent gray.



Figure 4.21: Same as in Fig. 4.20 but with $\mathbf{n} = (-1, 0, 0)$.

Operations over a non-convex polyhedron with disjoint regions

VOFTools also supports non-convex polyhedra with disjoint regions. For these geometries, the retained-volume function $v(c)$ remains continuous but may contain constant intervals when the cutting plane traverses gaps between regions. Figs. 4.20 and 4.21 show the liquid volume regions (blue color) obtained from the intersection between the VOFTools logo cell (routine `voftools_logo`; see Fig. 3.2), where each letter is represented by a disjoint polyhedral region, and half-spaces defined by planes given by $\mathbf{n} \cdot \mathbf{x} + c = 0$ with, respectively, $\mathbf{n} = (x_{nc} = 0, y_{nc} = -1, z_{nc} = 0)$ and $\mathbf{n} = (x_{nc} = -1, y_{nc} = 0, z_{nc} = 0)$, and the constant c obtained by solving the VCE problem for different liquid volume fractions f (the original cell is depicted on each picture using partial transparent gray). For the orientations in Figs. 4.20 and 4.21, the cutting planes are $y = c$ and $x = c$, respectively, and the retained regions satisfy $y < c$ and $x < c$. For the orientation used in Fig. 4.21, gaps between the letters produce constant intervals in $v(c)$ (Fig. 4.22). A target volume corresponding to such an interval admits multiple interface positions.

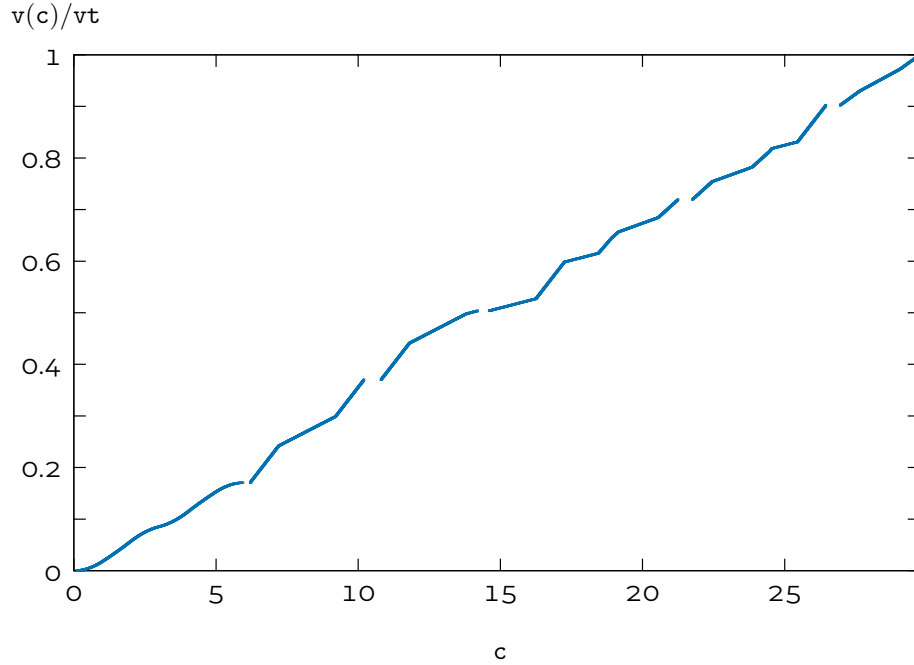


Figure 4.22: Liquid volume fraction $v(c)/v_t$ as a function of c for the VOFTools logo cell and an interface orientation given by $\mathbf{n} = (-1, 0, 0)$.

Volume fraction initialization

The accuracy of several initialization operations, performed by calling the routines `initf2d` (Section 3.29) for 2D and `initf3d` (Section 3.19) for 3D cases, is also as-

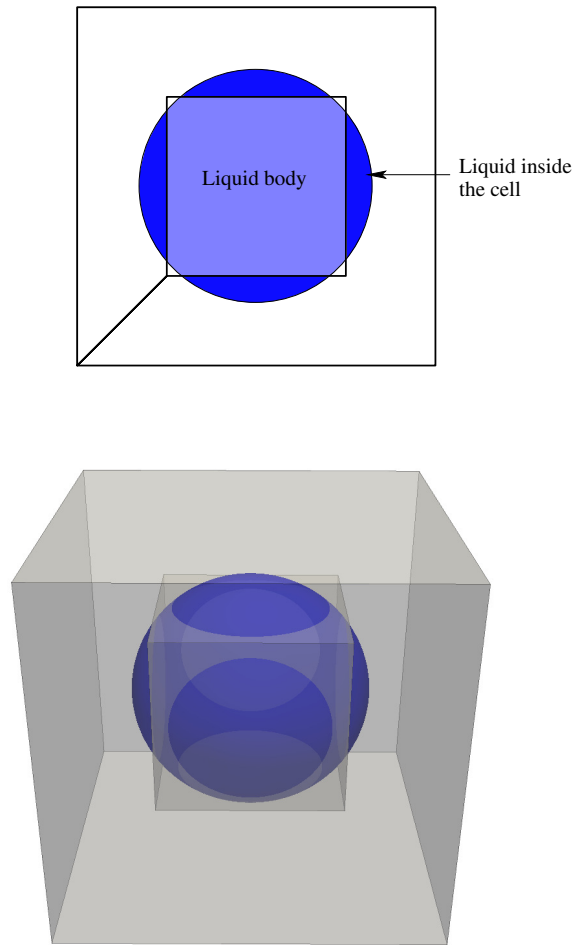


Figure 4.23: Liquid volume initialization of a circular and spherical liquid body (blue color) with radius $r_l = 0.325$ in the non-convex cell of Fig. 3.1 defined by the routine `voftools_hsquare` (top picture) and the non-convex cell of Fig. 3.2 defined by the routine `voftools_hcube` (bottom picture).

sessed. Firstly, the following two cases with non-convex cells (see Fig. 4.23) are considered:

1. Initialization of a circular liquid body with radius r_l centered at the center of the cell in a unit-length square with a half-length square hole located at its center (top picture in Fig. 4.23). The exact area fraction of liquid inside the cell can be obtained as

$$\mathbf{vf}_{\text{exact}} = \frac{2^3}{3} r_l^2 (\theta - \sin \theta),$$

where $\theta = 2 \arccos [1/(2^2 r_l)]$ and $2^{-2} \leq r_l \leq 2^{-3/2}$.

2. Initialization of a spherical liquid body with radius r_l centered at the center of the cell in a unit-length cube with a half-length cubic hole located at its center (bottom picture in Fig. 4.23). The exact volume fraction of liquid inside the cell can be obtained as

$$\mathbf{vf}_{\text{exact}} = \frac{2^4}{7} \pi (r_l - 2^{-2})^2 (2r_l + 2^{-2}),$$

with $2^{-2} \leq r_l \leq 2^{-3/2}$.

Fig. 4.24 shows the initialization error

$$|\mathbf{vf} - \mathbf{vf}_{\text{exact}}|$$

as a function of the number of divisions $\mathbf{nc}$ for these 2D and 3D cases with $r_l = 0.325$ (both liquid bodies are defined by calling the functions `func2d1` and `func3d1`). For these and the remaining initialization tests, the threshold parameter `tol` is chosen sufficiently large to bypass the preliminary full/empty classification and activate Cartesian subdivision (see Reference [10] for a more detailed discussion about this threshold parameter). It can be observed that the results tend to reach the exact solution with a second-order convergence rate.

The accuracy of the initialization procedure for more complex cases, for which obtaining the exact solution analytically is not an easy task, has been obtained using the Richardson extrapolation method [2]. Two material bodies with elliptical and toroidal interfaces defined by calling the functions `func2d2` and `func3d2` are given as a function of x, y, z -coordinates by, respectively,

$$f(x, y) = 1 - \left[\left(\frac{x - 0.5}{0.5} \right)^2 + \left(\frac{y - 0.5}{0.2} \right)^2 \right] \quad (4.1)$$

and

$$f(x, y, z) = \left(\frac{1}{3} \right)^2 - \left\{ \frac{2}{3} - [(x - 0.5)^2 + (z - 0.5)^2]^{0.5} \right\}^2 - (y - 0.5)^2. \quad (4.2)$$

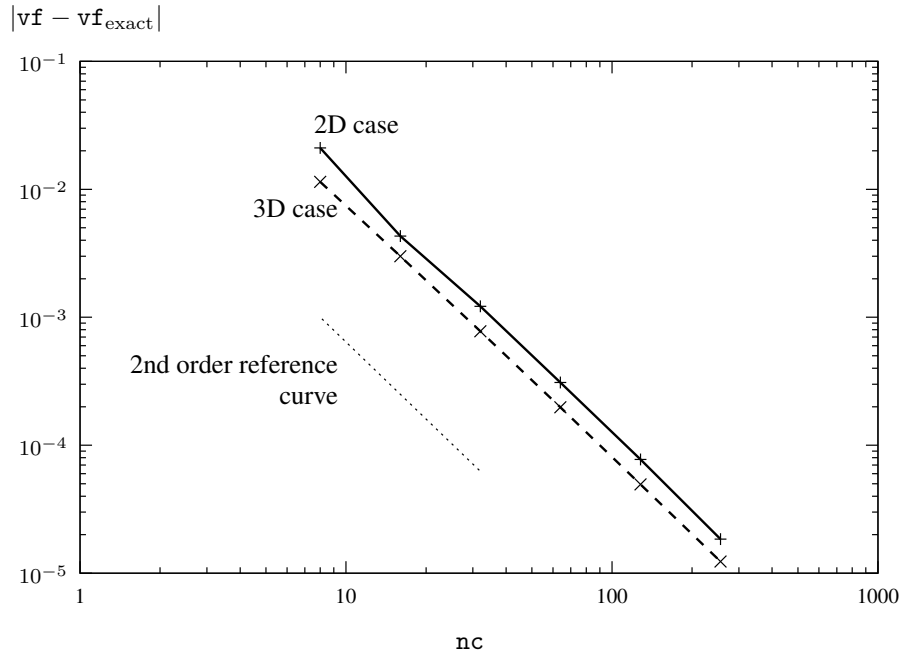


Figure 4.24: Initialization errors as a function of nc for the 2D and 3D cases of Fig. 4.23.

Fig. 4.25 shows the above material bodies along with several cells of Figs. 3.1 (left column) and 3.2 (right column). The accuracy is quantified assuming a leading second-order discretization error through the error defined as

$$|vf - vf_{\text{ext}}|,$$

where vf_{ext} is the extrapolated numerical solution [2] obtained as

$$vf_{\text{ext}} = \frac{4}{3}vf_{1024} - \frac{1}{3}vf_{512}$$

where vf_{1024} and vf_{512} are numerical results obtained on finer grids. The error curves in Fig. 4.26 exhibit approximately second-order behavior over the displayed refinement range, using the extrapolated solutions as numerical references.

4.2 PPIC operations

The PPIC testing suite is divided into two primary components:

- (1) **Surface fitting:** Programs designed to obtain a paraboloid that volumetrically best fits a set of input polygons. This includes `testfit.f90`, `testfit.c`, and `testfit.py`.
- (2) **Volume truncation and shifting:** Programs designed to compute the volume of the polyhedral approximation resulting from the intersection between a

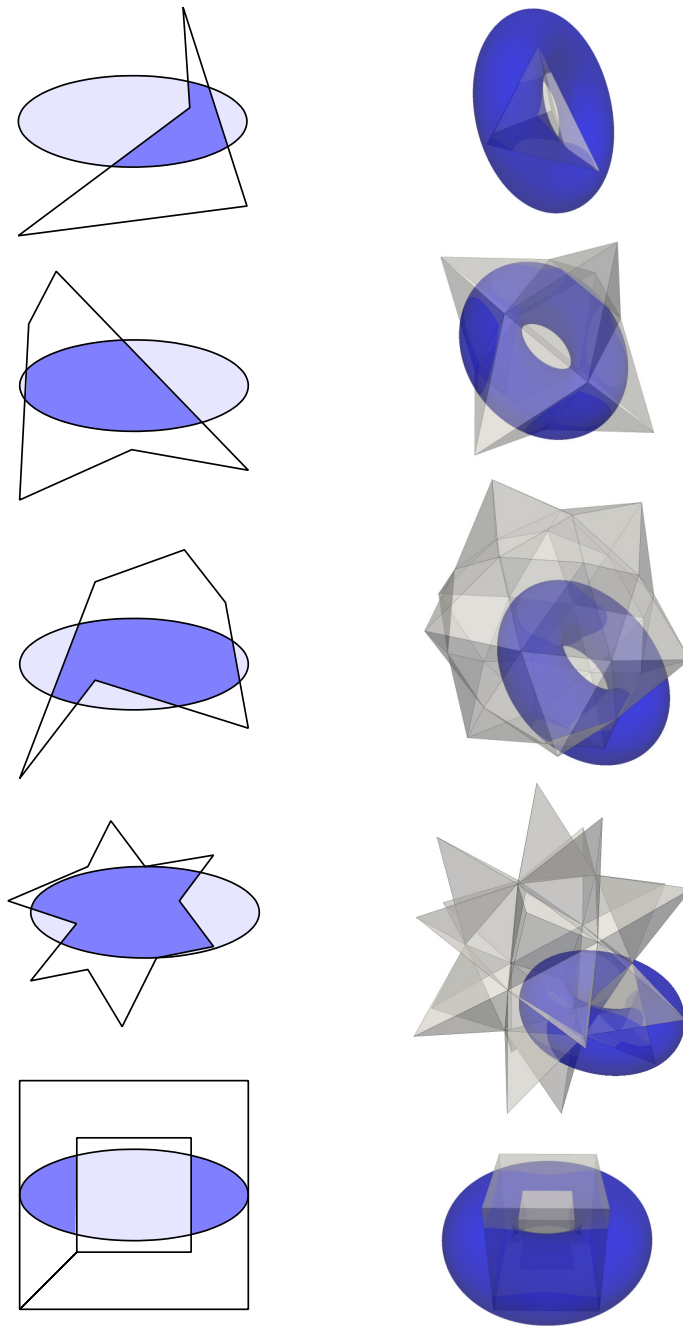


Figure 4.25: Examples for the initialization of the material bodies of Eq. (4.1) (left column) and Eq. (4.2) (right column) in several cells of Fig. 3.1 and Fig. 3.2, respectively.

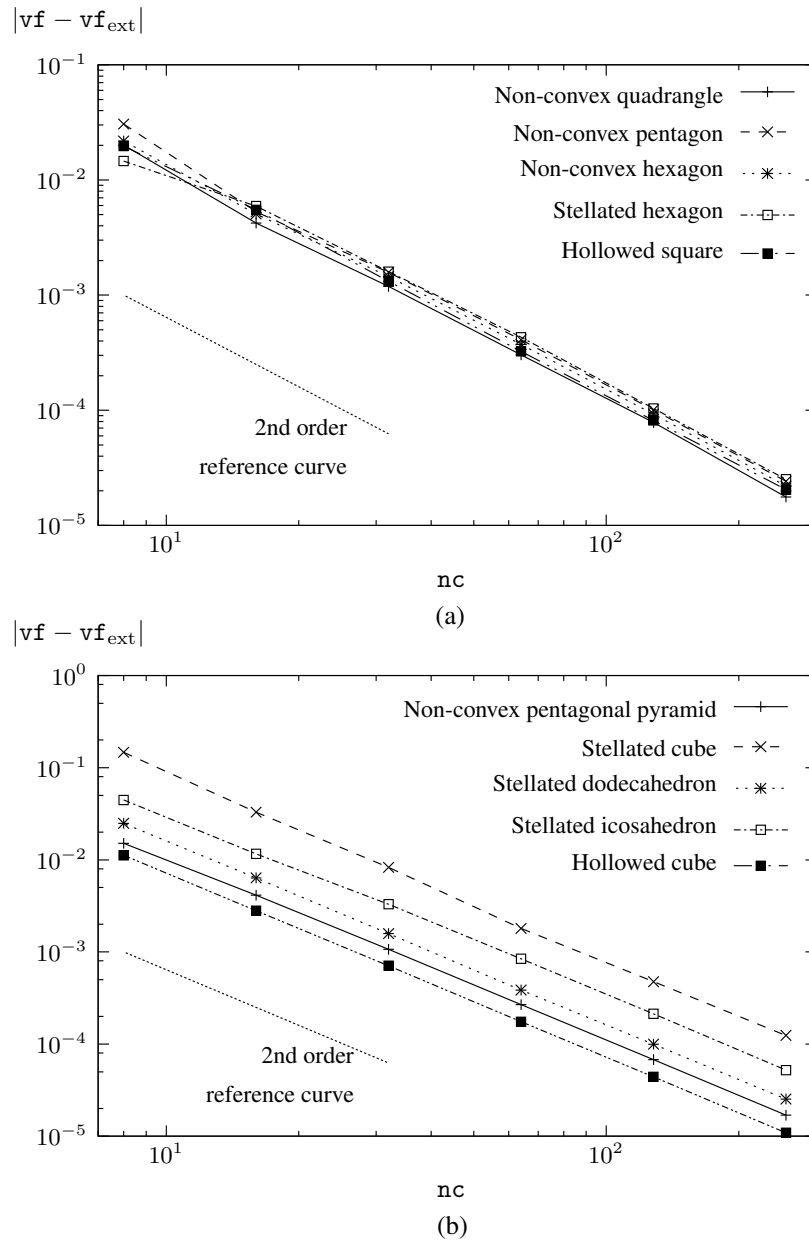


Figure 4.26: Initialization errors as a function of nc for several non-convex cells of Figs. 3.1 and 3.2 and the material bodies with (a) elliptical shape of Eq. (4.1) and (b) toroidal shape of Eq. (4.2), respectively.

paraboloid and an arbitrary polyhedral cell, and shift the paraboloid position to enforce discrete volume conservation. This includes `testparab.f90`, `testparab.c`, and `testparab.py`.

The configuration and execution of the test programs are described in Sections 4.2.1 for the fitting operation, and in Section 4.2.2 for the volume truncation and shifting operations. For a comprehensive assessment of the PPIC operations, refer to Reference [14].

4.2.1 Configuration and execution of the fitting operation

In this program, the user must specify the origin (`vp0(1)`, `vp0(2)`, `vp0(3)`) and the orthonormal basis matrix

$$\begin{pmatrix} \text{vn0}(1,1), \text{vn0}(1,2), \text{vn0}(1,3) \\ \text{vn0}(2,1), \text{vn0}(2,2), \text{vn0}(2,3) \\ \text{vn0}(3,1), \text{vn0}(3,2), \text{vn0}(3,3) \end{pmatrix}$$

defining the local coordinate system.

Additionally, the input parameters include the total number of polygons (`npol`) utilized for the paraboloid fitting and the vertex coordinates (`xv(i,j)`, `yv(i,j)`, and `zv(i,j)`) corresponding to vertex `j` of polygon `i`. All these geometric data must be expressed in the global Cartesian coordinate system and are provided in a text file named `polygons-set.txt`, which is automatically read by the program at runtime.

The program outputs the coefficients `coef = [c1, c2, c3, c4, c5, c6]` of the fitted paraboloid, defined by Eq. (3.1) within the local reference system, along with the sum of squared volumetric residuals (`errfit`) evaluated using the fitted coefficients.

The input data also include the number of polygons `npol`, the number of vertices `niv(i)` and unit normal components `xn(i)`, `yn(i)`, `zn(i)` of each polygon, and the coordinates `xv(i,j)`, `yv(i,j)`, `zv(i,j)` of vertex `j` of polygon `i`. Coordinates and normal vectors are expressed in the global Cartesian system. These data, together with the local reference system, are read from `polygons-set.txt` in the current working directory. Finally, a Gnuplot script (`fitted-paraboloid.gnuplot`) is generated to visualize the fitted paraboloid and the set of polygons within the local reference system.

Example. This example demonstrates the execution of the paraboloid fitting routine using a predefined set of polygons. The input data is provided in the file named `polygons-set.txt`, located in the data directory. This file is copied to the working directory and processed by the program, yielding the following results:

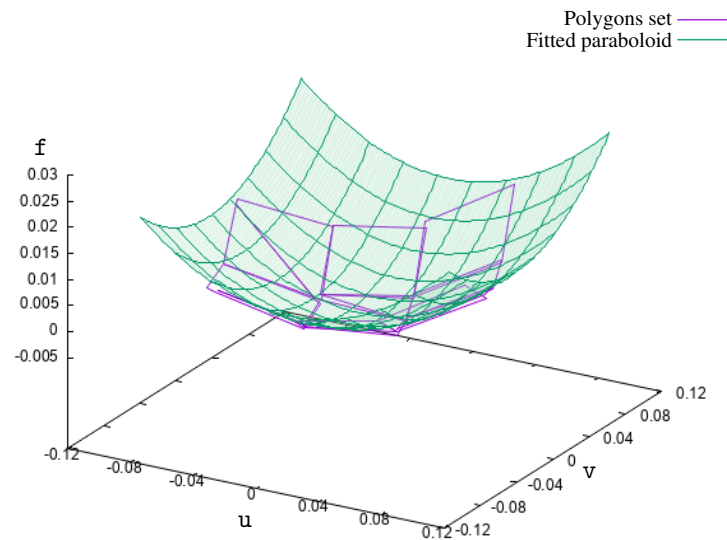


Figure 4.27: Example of the fitting operation performed by the test program `testfit`. The paraboloid (green surface) is fitted to a set of polygons (magenta lines) defined in a local coordinate system.

Paraboloid coefficients	
coef(1)	-0.0004938819558816568
coef(2)	-0.010011282362152601
coef(3)	0.04436260142637836
coef(4)	1.453899072370075
coef(5)	0.0031810769178630947
coef(6)	1.449920493501572
Fitting error	
errfit	8.2316009696390840e-15

The program also generates two additional output files:

- `polygons-set-lrs.txt`: Contains the coordinates of all polygon vertices transformed into the local reference system.
- `fitted-paraboloid.gnuplot`: A Gnuplot script for visualizing the fitted paraboloid and the input polygons in the local coordinate frame.

To visualize the results, execute Gnuplot in a terminal and load the script:

```
$ gnuplot
gnuplot> load 'fitted-paraboloid.gnuplot'
```

The resulting plot is shown in Fig. 4.27.

4.2.2 Configuration and execution of volume truncation and shifting operations

In this test program, the user must specify the coefficients of the paraboloid, along with the orientation and origin of the local coordinate system. These parameters are stored in the array `cparab` (length 12), where:

- The first six elements (c_1 – c_6) represent the coefficients of the paraboloid equation within the **local** coordinate system.
- Elements 7-9 specify the components of the unit vector along the local f -axis, expressed in the global coordinate system. Elements 10-12 specify the global coordinates of the local origin.

Additionally, the program requires the definition of the polyhedron (in the global coordinate system) used for volume computation and shifting operations. This geometry can be defined in two ways:

1. By passing the scalar value `icelltype` to the routine `defpol3d` (see Table 3.4 for predefined cell types).
2. By manually constructing the polyhedron vertices and faces.

Example. This example demonstrates the volume truncation and shifting operations for a paraboloid defined in a local coordinate system. The paraboloid equation is given by:

$$f(u, v) = 4u^2 + 4v^2,$$

where (u, v) are the local coordinates. The input array `cparab` (length 12) is configured as follows:

<code>cparab(1)</code>	0.0
<code>cparab(2)</code>	0.0
<code>cparab(3)</code>	0.0
<code>cparab(4)</code>	4.0
<code>cparab(5)</code>	0.0
<code>cparab(6)</code>	4.0
<code>cparab(7)</code>	0.0
<code>cparab(8)</code>	0.0
<code>cparab(9)</code>	1.0
<code>cparab(10)</code>	0.5
<code>cparab(11)</code>	0.5
<code>cparab(12)</code>	0.5

Here, the first six elements correspond to the coefficients of the paraboloid ($c_4 = 4$, $c_6 = 4$, others zero), and the last six elements define the local coordinate system origin (0.5, 0.5, 0.5) and orientation vector (0, 0, 1) in the global Cartesian frame.

The test case utilizes a cubic cell defined by `icelltype = 11` via the `defpol3d` routine. The simulation parameters are:

- Refinement parameter: $nc = 4$.
- Target fluid volume fraction: 0.5, corresponding to a target volume of $vf = 0.5vt$.

The program computes the volume of the polyhedral approximation of the region of intersection between the paraboloid and the cubic cell. It then shifts the paraboloid to enforce discrete volume conservation. The output includes:

1. The computed volume corresponding to the initial paraboloid (~ 0.08216).
2. The shifted coefficients of the paraboloid, exported to a text file named `cparab.dat`. Note that only one coefficient, c_1 , is modified to satisfy the volume constraint, resulting in a new value of $c_1 \approx -0.66263$.
3. The geometry of the polyhedron transformed into the local coordinate system, exported to a VTK file named `pol00000.vtk`.

The Python script `cellparabout.py`, located in the `src` directory, facilitates the visualization of the results. The visualization script requires NumPy and PyVista. Copy `cellparabout.py` from the `src` directory to the working directory containing the files `cparab.dat` and `pol00000.vtk`, and execute:

```
python cellparabout.py
```

The script generates a PDF file named `cellparabout.pdf` (see Fig. 4.28(b); Fig. 4.28(a) corresponds to the equivalent result for the initial paraboloid).

Furthermore, Fig. 4.29 presents analogous results for tetrahedral and stellated cubic cells at fluid volume fractions $v/vt = 0.05, 0.25, 0.5, 0.75$, and 0.95 , illustrating the application of the shifting procedure to both convex and non-convex geometries.

Test coverage. The test programs presented in this chapter exercise selected routines of the VOFTools library and illustrate representative applications. They do not constitute an exhaustive test suite covering every routine, input configuration, or geometric degeneracy. When applying the library to other configurations, users should follow the requirements documented for each routine and verify the results against suitable reference cases.

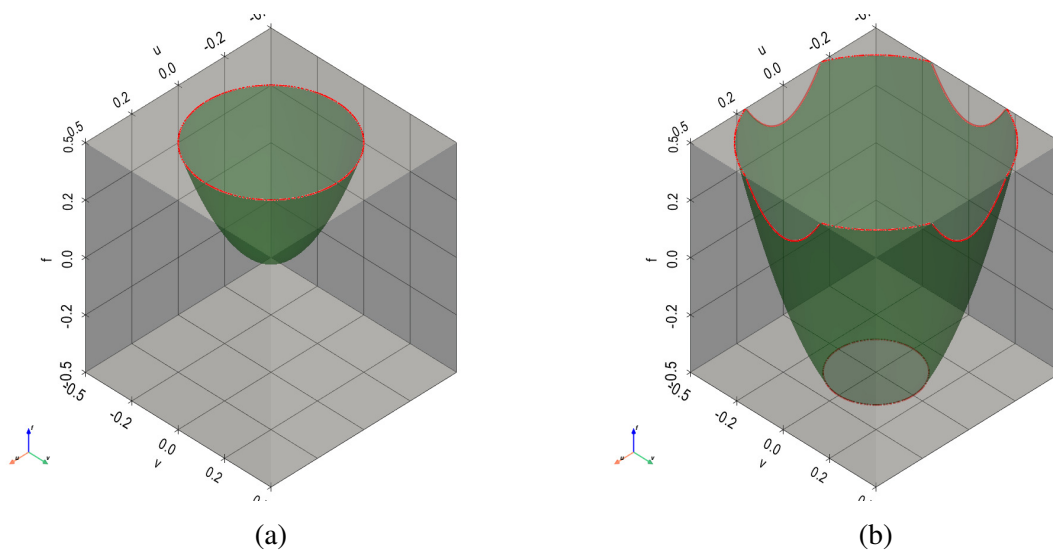


Figure 4.28: Example of the shifting operation performed by the test program `testparab`. (a) Before shifting: Initial paraboloid (transparent green surface) and cubic cell (transparent gray surfaces) in the local coordinate system (u, v, f) . (b) After shifting: Equivalent result for the shifted paraboloid. The intersection between the paraboloid and the cubic cell boundaries is represented by red lines.

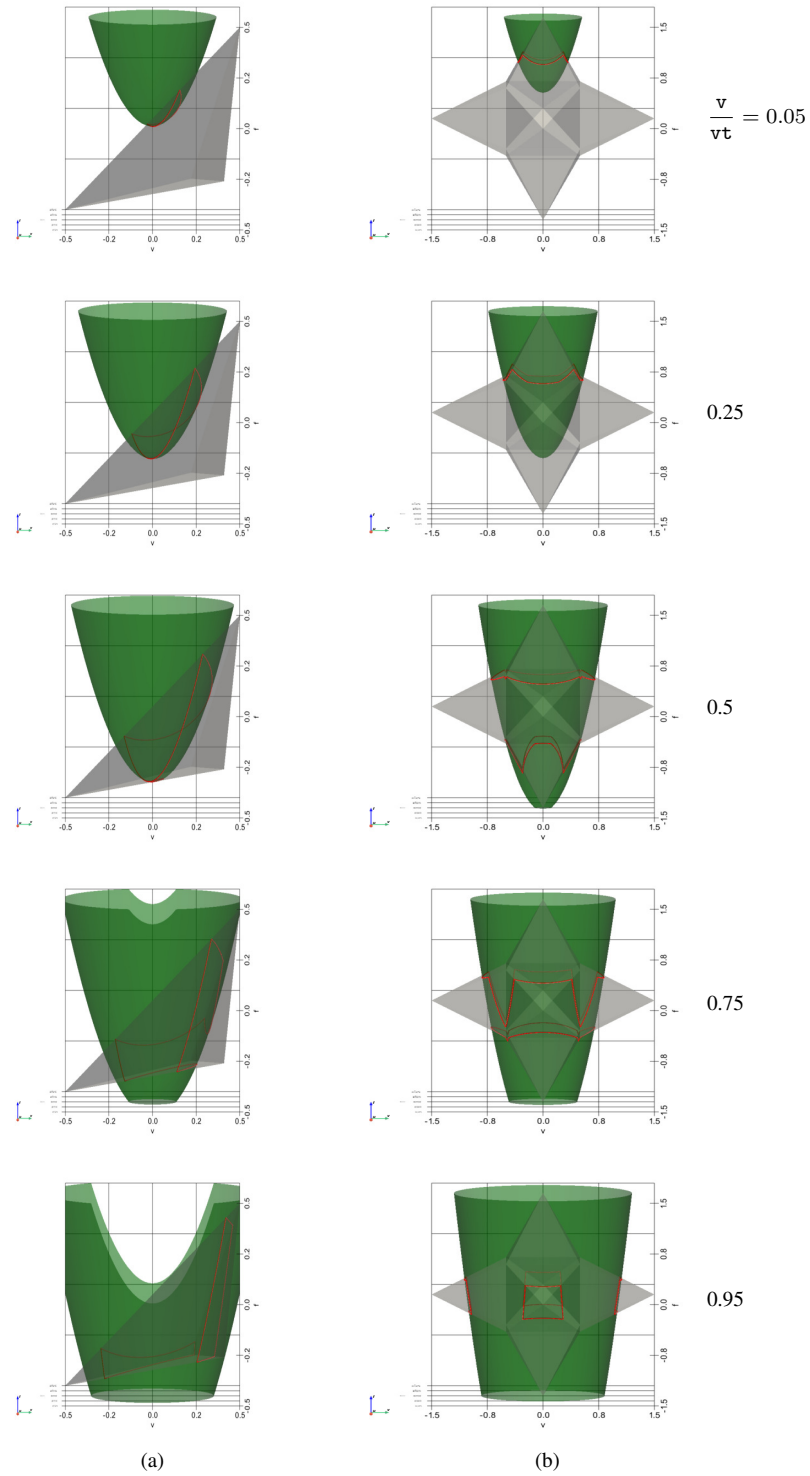


Figure 4.29: Equivalent to Fig. 4.28(b) but for (a) tetrahedral ($icelltype = 13$) and (b) stellated cubic ($icelltype = 113$) cells, and different fluid volume fractions v/vt .

Bibliography

- [1] H.T. Ahn, M. Shashkov, Multi-material interface reconstruction on generalized polyhedral meshes, *J. Comput. Phys.* 226 (2007) 2096-2132. <https://doi.org/10.1016/j.jcp.2007.06.033>.
- [2] I. Celik, W. Zhang, Calculation of numerical uncertainty using Richardson extrapolation: Application to some simple turbulent flow calculations, *ASME J. Fluids Eng.* 117 (1995) 439-445. <https://doi.org/10.1115/1.2817281>.
- [3] A. Henderson, J. Ahrens, C. Law, The ParaView guide (2004).
- [4] Z. Jibben, N.N. Carlson, M.M. Francois, A paraboloid fitting technique for calculating curvature from piecewise-linear interface reconstructions on 3D unstructured meshes, *Comput. Math. Appl.* 78 (2019) 643-653. <https://doi.org/10.1016/j.camwa.2018.09.009>.
- [5] Kitware, VTK – The Visualization Toolkit, <https://www.vtk.org>
- [6] J. López, J. Hernández, Analytical and geometrical tools for 3D volume of fluid methods in general grids, *J. Comput. Phys.* 227 (2008) 5939-5948. <https://doi.org/10.1016/j.jcp.2008.03.010>.
- [7] J. López, P. Gómez, J. Hernández, F. Faura, A two-grid adaptive volume of fluid approach for dendritic solidification, *Comput. Fluids* 86 (2013) 326-342. <https://doi.org/10.1016/j.compfluid.2013.07.017>.
- [8] J. López, J. Hernández, P. Gómez, F. Faura, A new volume conservation enforcement method for PLIC reconstruction in general convex grids, *J. Comput. Phys.* 316 (2016) 338-359. <https://doi.org/10.1016/j.jcp.2016.04.018>.
- [9] J. López, J. Hernández, P. Gómez, F. Faura, VOFTools – A software package of calculation tools for volume of fluid methods using general convex grids, *Comput. Phys. Commun.* 223 (2018) 45-54. <https://doi.org/10.1016/j.cpc.2017.09.032>.
- [10] J. López, J. Hernández, P. Gómez, F. Faura, Non-convex analytical and geometrical tools for volume truncation, initialization and conservation enforcement in VOF methods, *J. Comput. Phys.* 392 (2019) 666-693. <https://doi.org/10.1016/j.jcp.2019.04.055>.

- [11] J. López, J. Hernández, P. Gómez, C. Zanzi, R. Zamora, VOFTools 3.2: Added VOF functionality to initialize the liquid volume fraction in general convex cells, *Comput. Phys. Commun.* 245 (2019) 106859. <https://doi.org/10.1016/j.cpc.2019.07.022>.
- [12] J. López, J. Hernández, P. Gómez, C. Zanzi, R. Zamora, VOFTools 5: An extension to non-convex geometries of calculation tools for volume of fluid methods, *Comput. Phys. Commun.* 252 (2020) 107277. <https://doi.org/10.1016/j.cpc.2020.107277>.
- [13] J. López, J. Hernández, On polytope intersection by half-spaces and hyperplanes for unsplit geometric volume of fluid methods on arbitrary grids, *Comput. Phys. Commun.* 300 (2024) 109167. <https://doi.org/10.1016/j.cpc.2024.109167>.
- [14] J. López, Unsplit geometric volume-of-fluid method with iterative piecewise-paraboloid interface reconstruction on arbitrary three-dimensional grids, *J. Comput. Phys.* 553 (2026) 114714. <https://doi.org/10.1016/j.jcp.2026.114714>.
- [15] J. López, VOFTools 6: Improved performance and added functionality for piecewise-paraboloid interface calculations, submitted to *Comput. Phys. Commun.*
- [16] E.A. Merritt, et al., Gnuplot. An interactive plotting program. <http://www.gnuplot.info/>
- [17] NumPy Developers, F2py user guide and reference manual, <https://numpy.org/doc/stable/f2py/index.html> (accessed 8 october 2026).
- [18] Python Package Index (PyPI), <https://pypi.org>.
- [19] P. Peterson, F2py: a tool for connecting Fortran and Python programs, *Int. J. Comput. Sci. Eng.* 4 (4) (2009) 296-305. <https://www.inderscience.com/info/inarticle.php?artid=29165>.
- [20] R. Scardovelli, S. Zaleski, Analytical relations connecting linear interfaces and volume fractions in rectangular grids, *J. Comput. Phys.* 164 (2000) 228-237. <https://doi.org/10.1006/jcph.2000.6567>.